
DESCRIPTION LOGIC PROGRAMS REVISITED: THE BENEFITS AND LIMITS OF SPECIALIZED ONTOLOGY REASONING

PREPRINT

Raphael Volz
Pforzheim University

9 September 2026

ABSTRACT

Restricting the expressive power of a knowledge representation language can make reasoning more efficient, but the conditions under which this advantage appears require empirical examination. We revisit Description Logic Programs, which connect ontology languages with rule-based inference, through a reconstruction of the original work and experiments on changing knowledge and query answering. We investigate whether sharing explanations and adapting the order of reasoning reduce computation while preserving its conclusions. Sharing explanations accelerates a favourable update task by approximately 7.5 times, with a similar result in a separate replication, but adds 18–19% overhead when suitable explanations are absent. Adaptive evaluation reduces some intermediate work without consistently improving complete task times. Comparisons with established reasoners likewise reveal operation-specific advantages and substantial disadvantages. The original Bach examples also exposed a loss of valid conclusions during deletion in the tested ZodiacEdge rule reasoner. On a finite task involving the existence of unnamed individuals, specialized reasoning is 2.6–21.5 times faster than the tested HermiT distribution; the task also belongs to the established OWL 2 EL profile. These findings support a conditional account of specialization: benefits depend on the structure of the knowledge, the change, and the question. A fragment’s theoretical or computational value is distinct from how frequently it is used. The evidence supports useful specializations, without establishing a general performance advantage or a new complexity bound.

Keywords Ontology reasoning · Description Logic Programs · knowledge representation · incremental reasoning · experimental evaluation

1 Introduction

A knowledge base contains more than a collection of observations. Statements about categories and relationships allow further conclusions to be drawn. In the thesis’s Bach family example, Johann Ambrosius can be identified as a father even though the description does not name his child. An ontology reasoner makes such implications explicit. The challenge is to provide useful expressive power while keeping inference manageable, especially when the observations or the governing statements change.

Description Logic Programs (DLP) address this challenge by connecting a restricted ontology language with rule-based inference [7]. The 2004 thesis *Web Ontology Reasoning with Logic Databases* developed this connection into reasoning services and methods for maintaining inferred knowledge [31, 32]. Over the following two decades, research advanced the treatment of alternative explanations, shared computation, and the order of inference. Standardized OWL 2 profiles also established several language restrictions intended for different reasoning needs [18].

These developments raise a scientific question that is broader than whether an old implementation can be made faster: under what conditions does specialization remain useful? A restricted language can offer a mathematical advantage even when its practical prevalence is unknown. A particular reasoner can also outperform a broader one on a defined

task without demonstrating a new mathematical separation. Neither kind of evidence alone establishes that the historical language interface is the best choice for all applications.

We examine three questions. First, when can explanations shared across many individuals reduce the cost of keeping conclusions up to date? Second, when does avoiding intermediate reasoning work reduce the time required for a complete task? Third, can reasoning within an expressive DLP fragment offer computational value relative to a broader OWL reasoner, independently of how often applications use that fragment alone?

The study combines semantic reconstruction, controlled experiments, and comparisons with external reasoners. It includes deliberately unfavourable cases and reports unsuccessful interventions. The research questions organize an exploratory programme developed in stages; they are not presented as a retrospectively preregistered experiment. A focused review of the original specification establishes the meaning the reconstruction must preserve. Constructed workloads then isolate proposed mechanisms, while established university and database workloads test selected tasks beyond those examples.

The contribution is an empirical reassessment of DLP and an account of the conditions under which the examined specializations help or fail. The original connection between ontology languages and rules, support-aware maintenance, and adaptive query evaluation remain attributable to earlier work. The new evidence concerns the tested combination of shared explanations, bounded adaptation, and matched reasoning tasks. It provides preservation arguments and evidence against unconditional performance expectations, rather than claiming a generally superior reasoning algorithm.

2 Background and related research

An ontology describes categories, relationships, and constraints within a domain. An ontology reasoner determines what follows from these descriptions and the available observations. We use the thesis’s original Bach family knowledge base as a running illustration [31, Table 2.5, p. 35]. It describes Johann Ambrosius as a man with a male child, without naming that child. It also states that men are persons and defines a father as a man with a child who is a person. A reasoner can therefore conclude that Johann Ambrosius is a father without identifying his child. This distinguishes an implied conclusion from an explicitly recorded statement.

Missing information is also different from falsehood. The same knowledge base names Wilhelm Friedemann as Johann Sebastian’s child. As the thesis explains, this does not establish that he is Johann Sebastian’s only child [31, Example 5.4.1, p. 131]. Incomplete family records do not rule out additional children.

A complementary illustration comes from the Lehigh University Benchmark (LUBM), used in our external comparisons [8, 16]. Its ontology states that every graduate student is a person who takes some graduate course, that graduate courses are courses, and that people taking courses are students. A graduate-student declaration therefore implies student membership even when it names no course. This distinction matters for LUBM’s sixth query, which asks for all students. Bach connects the present study to the original work; LUBM connects the same reasoning ideas to an established benchmark.

The Web Ontology Language, OWL, gives such statements a precise meaning [19, 26]. Greater expressive power permits more complex descriptions, but can make inference more demanding. A central research question is therefore which restrictions preserve useful forms of expression while enabling simpler reasoning. A *fragment* is a language obtained by such restrictions. Its value can arise from a mathematical guarantee, a computational advantage on a specified task, or both.

Description Logic Programs were introduced as a connection between ontology languages and rules of the form “if these conditions hold, then this conclusion follows” [7]. Volz’s 2004 thesis developed this connection into methods for answering questions and maintaining inferred knowledge as facts and rules change [31]. The original research already addressed changes to the ontology’s rules, rather than only changes to observations [32]. The work examined here inherits that problem and semantic foundation. Its expressive mode, called DLP L3, also admits statements requiring an individual to exist without naming them. The thesis illustrates this using Johann Ambrosius’s unnamed child [31, Example 4.5.2, p. 94]. This additional capability requires separate attention to whether a reasoning procedure terminates.

One influential approach computes consequences in advance and retains them for later questions. This is called *materialization*. Its main attraction is that repeated questions can reuse earlier deductions. Its main difficulty is maintaining those deductions when their premises change. Adding a new premise usually creates further conclusions. Removing one is subtler: a conclusion may have several independent justifications. The thesis’s extended Bach family tree supplies two paths from Johannes to Wilhelm Friedemann, one through Johann Sebastian and another through Maria Barbara. In its hypothetical withdrawal of the Johann Sebastian–Wilhelm Friedemann link, Johannes remains an

inferred ancestor through Maria Barbara [31, Section 6.2.1 and Figure 6.2, p. 150]. The example changes the recorded knowledge, rather than making a historical claim about parentage.

Research has developed this idea in several directions. Deletion and rederivation, established before the thesis, first withdraw potentially affected conclusions and then restore those still justified [9]. Subsequent methods examine alternative proofs more selectively, count suitable forms of support, or combine specialized procedures [11, 13, 22, 23]. Changes to the rules themselves have also received renewed attention [15, 33]. These studies establish the broader principle that maintenance should depend on the structure of the affected reasoning, not simply on the number of changed statements.

A second direction reduces repeated work across similar deductions. Type-based reasoning groups individuals with recurring descriptions, while compressed reasoning represents repeated patterns more economically [2, 12]. Parallel and column-oriented systems organize computation and storage to handle larger knowledge bases [5, 21]. More recent work further specializes representations for common patterns [35]. These approaches motivate our investigation of whether individuals with the same current starting facts can share the work of establishing a surviving justification. The general idea of sharing proofs is inherited; the present study tests a restricted form of it during simultaneous changes to facts and rules.

A third direction concerns the order in which conditions are tested. Combining records that satisfy several conditions is known as a *join*. An unfavourable order can produce many temporary combinations that later tests discard. Research has established stronger algorithms for combining multiple relations [24], and recent studies investigate robust filtering and the avoidance of unnecessary intermediate expansion [3, 27]. These findings motivate our experiments with limited adaptation of evaluation order. Their mathematical guarantees do not automatically apply to the simpler strategy examined here.

Finally, OWL 2 profiles formalize different restrictions for different reasoning needs. The rule-oriented RL profile explicitly acknowledges DLP among its influences [18]. This historical continuity motivates reassessment rather than a claim that the earlier fragments have become obsolete. Our contribution is a corrected reconstruction and a controlled investigation of when selected restrictions and reasoning strategies help, including cases in which their costs outweigh their benefits.

3 Study design

The study developed in stages. We first examined two ways of reducing repeated reasoning after changes to a knowledge base, then used the observed costs and the literature review to select further experiments. The later questions and comparisons were therefore exploratory. Protocols were recorded before their respective timing series, but the research programme was not publicly preregistered. Earlier trials, unsuccessful cases, and subsequent replications are retained in the appendix evidence.

3.1 Interventions and comparisons

The first intervention changes how groups of records satisfying a rule’s conditions are combined. One option first collects the records affected by a change; another starts with records meeting all the conditions and then checks which were affected. We investigated whether information about recent changes helps choose between these equivalent approaches, including the cost of the choice.

The second intervention shares explanations for conclusions that remain valid after a change. Several individuals may satisfy exactly the same relevant conditions. When they do, one calculation can establish which conclusions still follow for all of them. These explanations can prevent unnecessary withdrawal and restoration of conclusions. If no suitable explanation is found, the ordinary maintenance procedure still determines the answer. This restricted use of shared explanations builds on support-aware maintenance and shared reasoning [11–13, 22].

Follow-up changes reduced repeated work in three further ways: reusing checks that remain valid when information is added, updating access structures only where information has changed, and preparing common rule operations for repeated use. These changes apply established techniques; the experiment evaluates their combined effect. A later experiment reconsidered evaluation order by briefly examining likely consequences of alternative choices. This was motivated by recent work on avoiding large intermediate results [3, 27]. The examination changes the search order, without excluding possible answers.

We distinguish three comparisons. Controlled variants of the same system estimate an intervention’s effect. Comparisons with a preserved earlier version describe cumulative progress. Executing an independent reasoner on matched inputs

and tasks tests relative performance beyond the local system. Published timings on different machines provide context, but cannot supply speed ratios for our experiments.

3.2 Controlled and established workloads

The initial experiment combined the first two interventions in a factorial design: each was either present or absent, yielding four variants. Eighteen constructed cases varied the concentration of changes, surviving explanations, recursion, and opportunities to share reasoning. Nine repetition blocks tested all variants in fresh processes, with a rotating order. Complete update time was the primary outcome. A separate replication used new random seeds and 144 fresh processes to re-examine the main explanation-sharing benefit and its adverse cases. It supplements the original observations rather than replacing them.

External validation began with LUBM, which provides a university ontology, generated data, and reference answers [8, 16]. We used one generated university and retained the logical ontology. Three repetition blocks compared three local variants, and all fourteen queries were checked against the complete official answer sets. This tests established questions on a modest dataset; it is not a scalability study.

For changing rules, we used the positive university rule programme supplied with ZodiacEdge [33, 34]. Two scenarios added and removed designated sets of eight and sixteen rules. Four repetition blocks compared four local variants; a separate four-block comparison alternated our candidate and ZodiacEdge itself. Additional samples removed and restored 1,000 facts, following the update size used in earlier maintenance research [11]. Samples within one process are repeated observations, not independent process replicates. The author’s rule programme and the complete LUBM ontology are distinct reasoning tasks.

The Transaction Processing Performance Council’s Benchmark H (TPC-H) evaluates database systems answering analytical questions over business data [30]. Our query experiments used the conditions underlying its queries Q3, Q5, and Q9 at two data scales. These six components retain the combinations of input rows satisfying each query, but exclude aggregation and result ordering. They therefore do not constitute a full TPC-H evaluation or reproduce the larger study of Qiao and colleagues [27]. The first comparison used three blocks, with all three components evaluated sequentially in each fresh process. A follow-up gave each component its own process and compared three variants across three blocks. It added controls deliberately chosen to expose unhelpful decision costs and misleading intermediate patterns, giving 90 processes overall. The follow-up revisits the workload that motivated it; it does not provide independent evidence of generalization.

3.3 Expressiveness, correctness, and measurement

A separate experiment compared our reasoner with HermiT [6] on a finite family using existential statements: every member of one class must relate to some member of another. Both systems determined consistency and returned all named members of two target classes. Populations of 100, 1,000, and 10,000 individuals were tested in three blocks, comparing the candidate, its earlier baseline, and HermiT. These 27 processes test a selected DLP L3 task beyond the structural OWL 2 RL syntax. The family also belongs to OWL 2 EL [18], and its answers do not require explicitly constructing the unnamed individuals. It tests computational specialization, without assuming that such ontologies are common.

Correctness was checked separately from performance. Update experiments compared complete conclusions with fresh recomputation; small cases also used independently written reference procedures. Query experiments checked exact answers against official LUBM answers or independent database evaluation. The existential experiment used independently justified answers and controls omitting each essential statement. Agreement of counts alone was insufficient.

A further comparison returned to the original Bach examples at their original scale [31, Table 2.5, p. 35; Figure 6.2, p. 150]. The current reasoner, its preserved earlier version, and HermiT answered 25 questions on the complete ontology. A separate family task asked nine questions, including the complete ancestor and dynasty relationships. All four systems evaluated the original and three changed family states from scratch; the two local versions and ZodiacEdge also applied and reversed each change while retaining state. Twelve repetition blocks specified 336 fresh processes, with rotating system order. The older version was measured anew: its saved historical results contained no Bach comparison. These small examples test source fidelity and update behaviour, without making a scaling claim.

For this comparison, the complete ontology’s existence requirements and constraints fall outside ZodiacEdge’s positive-rule task. Its family evaluation uses the same recorded relationships and rules. A wrong answer disqualifies the entire system–task combination from a timing ranking; passing executions are not selected to hide unsuccessful ones. Separate diagnostic checks investigate discrepancies while retaining the recorded implementation.

Timings include complete updates or complete consumption of query answers. Query computation and data preparation are reported separately, with combined costs where measured. HerMiT comparisons include loading, consistency, and answer retrieval because systems perform inference at different stages. Independent checks lie outside these task intervals. In controlled comparisons, ratios pair observations within repetition blocks and report their median, rather than dividing two separately calculated medians. Historical comparisons use ratios of the medians from separate runs.

All new measurements used one Apple M4 Max desktop. Process restarts did not clear filesystem caches. Unrelated activity may have overlapped the primary study; its extent is unknown, and the original samples were retained. Later timed studies were scheduled separately. Small repetition counts, one host, and selected workloads make the comparisons descriptive. Detailed protocols, timing boundaries, individual observations, and preservation arguments are provided in the technical appendices.

4 Results

4.1 A semantic basis for comparison

The reconstruction exposed consequential inconsistencies in particular printed formulas and procedures. For example, separate questions about the equivalence of two categories can interfere if their test assumptions are combined and individuals may denote the same object. Similarly, withdrawing a rule must remove conclusions whose only explanation depends on that rule. The technical appendices give explicit counterexamples and the required corrections. These findings concern the specification. The historical implementation was not examined, so its behaviour cannot be inferred from these findings.

The experiments evaluate the corrected interpretation. The complete official answer sets for all fourteen LUBM questions were recovered in every measured run. The earlier controlled update experiments matched the full conclusions of fresh recomputation, and the external query comparisons matched independently established answers. The later Bach comparison also records an external update discrepancy, discussed below. These checks support the interpretation of the measured tasks. They do not establish correctness for all possible ontologies or make the distinct benchmark tasks semantically interchangeable. Table 1 summarizes the principal comparisons and their different evidential roles.

4.2 Sharing explanations helps when explanations survive

In the favourable constructed case, many individuals shared starting facts and retained an independent reason for a conclusion after a change. Establishing that reason once avoided withdrawing and rebuilding the same kind of conclusion repeatedly. The median of paired update-time ratios was approximately 0.134, corresponding to a 7.45-fold improvement. A separate replication obtained a 7.56-fold improvement on the corresponding case.

When no suitable explanation survived, the search for one could not prevent the required removals. It nevertheless consumed time, producing approximately 18–19% overhead. The replication reproduced both the benefit and this cost. Cases with less opportunity to share explanations gave more limited benefits. Figure 1 shows the full range of the replication’s cases, including the adverse outcomes.

The result supports a mechanism with identifiable conditions. Reuse is valuable when the same valid reasoning applies to many individuals and avoids a substantial amount of repeated work. Attempted reuse can be harmful when those conditions are absent. The experiments therefore support conditional use, not an expectation that additional explanation checking always improves maintenance.

4.3 Less intermediate work does not guarantee a faster task

The first adaptive intervention changed how the sets satisfying a rule’s conditions were combined. In one case it avoided tens of thousands of intermediate visits. Across the ten cases, however, the geometric mean of paired complete-update ratios was 1.0014: there was no overall improvement. Every reported uncertainty interval included one. Much of the task’s cost lay outside the condition evaluation that the intervention changed. The study therefore does not establish a complete-task advantage from that reduction in intermediate work.

The subsequent query experiment examined a different source of repeated work: the order in which records satisfying several conditions were combined. On the larger TPC-H Q5 component, a brief look ahead reduced examined candidates from 621,901 to 155,969. The median paired query-computation ratio was 0.510, with an observed range of 0.481–0.520 across three repetitions. The computation itself was therefore approximately twice as fast in this case.

The benefit did not extend uniformly to the complete task. Its paired preparation-plus-query ratios ranged from 0.972 to 1.273, spanning both improvement and deterioration. Variation in preparation costs preceded query execution and

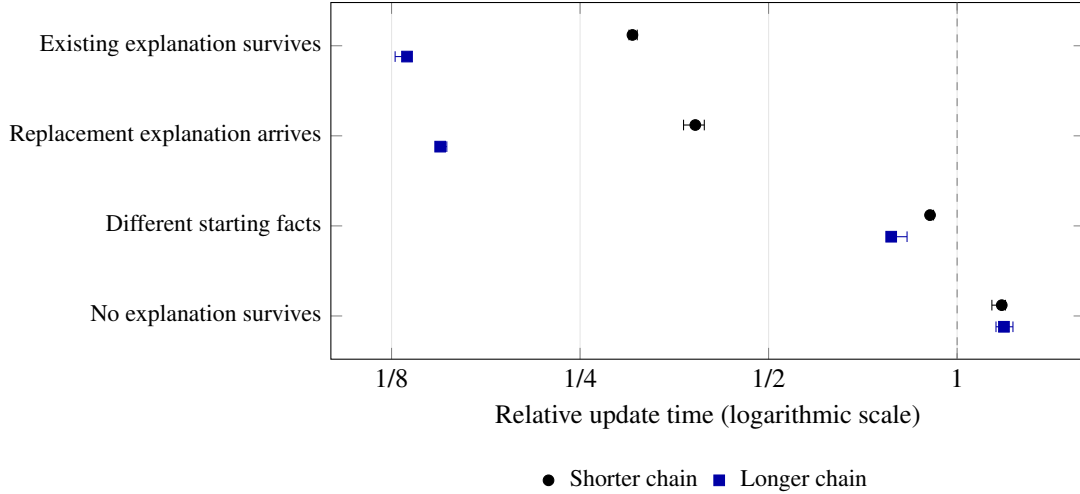


Figure 1: Sharing explanations reduces the cost of updating knowledge when conclusions remain justified, but adds work when their justification disappears. The experiment changes facts and rules in cyclic chains concerning 256 individuals; the shorter and longer chains have four and 32 links before the cycle closes. Points show median time with shared explanations divided by ordinary maintenance time, from nine paired repetitions per case in an independent replication. Bars show descriptive 95% paired bootstrap intervals (2,000 resamples). Values left of the dashed line indicate faster updating.

Table 1: The principal comparisons answer different scientific questions; they do not establish an overall ranking of reasoning systems. Performance changes are derived from median paired time ratios on the same machine. Query timings in the fourth row exclude data preparation.

Evidence	Comparison	Principal finding
University ontology	Full LUBM ontology and its 14 standard questions.	All reference answers recovered in every repetition.
Changes to rules	Established university rule program, compared with the reference version.	Rule additions $2.1\text{--}3.2 \times$ faster; removals $1.4\text{--}1.6 \times$ faster.
External rule reasoner	The same rule changes, compared with ZodiacEdge.	One addition $6.2 \times$ faster; one removal $41 \times$ slower.
Order of reasoning	A difficult TPC-H Q5 component and four deliberately adverse controls.	Q5 needs 49% less query time; controls need 7–24% more.
Broader OWL reasoner	Selected DLP L3 task, compared with HermiT at three data sizes.	$2.6\text{--}21.5 \times$ faster; the tested family also belongs to OWL 2 EL.

cannot be attributed to the ordering decision on this evidence. The four adverse controls became 6.6–23.8% slower in query computation. They retained the same number of examined candidates, exposing the cost of making decisions that did not reduce subsequent work. The other five database components showed no comparable reduction in candidates.

Together, the two experiments distinguish an algorithm’s local opportunity from the benefit of using it. Avoided combinations are a plausible mechanism, but total cost also includes deciding what to do and preparing the data. A promising reduction in one part of a task is insufficient evidence for a general speed advantage.

4.4 External comparisons reveal different strengths

The new system reduced the two designated rule-insertion times by factors of 3.18 and 2.08 relative to the preserved local reference version. This establishes progress against that reference, but an external comparison asks a different question. On the same university rule program, it was approximately 6.2 times faster than ZodiacEdge when inserting the eight-rule set. ZodiacEdge was faster on several other operations, including deletion of the sixteen-rule set, where our candidate took approximately 41 times as long. The corresponding median deletion times were 0.97 seconds for our candidate and 0.023 seconds for ZodiacEdge; the large relative difference concerns an operation taking less than a second in either system at this scale.

All conclusions agreed for these university rule changes. The difference is therefore a performance limitation on a matched task, not a trade of completeness for speed. A separate diagnostic examination found that repeated preparation and checking dominated the candidate’s cost in that deletion case. The number of withdrawn conclusions was exactly the number that should disappear. This observation identifies a direction for further investigation, but does not demonstrate how much of the external performance gap a future change would remove.

The final regression evaluation also preserved the less favourable local outcomes. All 54 workloads passed their correctness checks. Although all 51 comparable initial-inference medians were lower than the saved reference in that run, two of 36 update comparisons were slower, by 25.8% and 2.7%. Those historical runs were not interleaved experiments and cannot isolate the effect of the latest intervention. Neither local progress nor a selected external win establishes an overall ranking.

4.5 The original examples test preservation of meaning

The Bach comparison returns to the examples that motivated the reconstruction. It covers the complete ontology’s 25 questions and nine questions on the separate family graph, including all ancestor and dynasty relationships. These original examples are small: their timings describe the cost of answering and maintaining a particular set of questions, rather than performance on large family histories.

The full Bach ontology gave identical answers to all 25 questions in the three supported systems. Median complete-task time was 55.6 ms for the current reasoner, 89.5 ms for the preserved earlier implementation, and 347.4 ms for HermiT.

All four systems agreed on the complete family answers when each resulting state was computed from scratch. During incremental fact replacement, however, the pinned ZodiacEdge implementation failed exact-answer validation in every second (6/12) process. This task therefore receives no ZodiacEdge timing ranking. The current and earlier implementations passed every tested update and restoration. Detailed phase and process times are given in the technical appendices.

ZodiacEdge had lower task times on all four family reconstructions and both rule changes. For the original family, its median task time was 4.4 ms, compared with 7.5 ms for the current reasoner. Including process startup, checking and shutdown, the corresponding medians were 137.9 and 135.6 ms. The timing boundary therefore matters on these small inputs.

Independent checks of the recorded ZodiacEdge version exposed a limitation in one family update. After withdrawing the Johann Sebastian–Wilhelm Friedemann link and inserting another, some executions lost the conclusion that Johannes is an ancestor of Wilhelm Friedemann. The path through Maria Barbara still justifies that conclusion. Recomputing the changed family from its assertions provides a separate check of the required result. The discrepancy concerns the tested incremental operation in that version; it does not contradict the successful checks on the earlier university rule changes.

The affected system–task combination is excluded from performance rankings. Retaining this discrepancy is part of the comparison: an apparently quicker operation cannot count as an improvement when it loses a required answer. The evidence also distinguishes computing a changed state afresh from maintaining it through a sequence of changes.

4.6 A restricted language can offer computational value

The existential reasoning experiment examines the distinction between existence and naming illustrated by Johann Ambrosius’s unnamed child and LUBM’s unnamed course. It uses a separate, simplified family of constructed ontologies: belonging to one class requires a related individual to exist, and that relationship implies membership in another class. These measured inputs are distinct from both illustrative ontologies. The existential statement is essential to the tested conclusion; removing it changes the correct answer. Its form goes beyond the permitted syntax of OWL 2 RL.

Across the three populations, the candidate’s paired task-time ratios to HermiT were approximately 0.046, 0.263, and 0.378. These correspond to 21.5-, 3.8-, and 2.6-fold advantages for the selected tasks. At the largest population, median task times were 0.94 seconds for the candidate and 2.48 seconds for HermiT. The result persists when the entire fresh process is timed, including startup and completion: the corresponding ratios are approximately 0.379, 0.476, and 0.415. All systems agreed on consistency and the named individuals satisfying both questions. Most of the advantage was already present in the earlier local version, so it cannot be attributed to the latest optimizations.

This provides an example of useful computational specialization. It requires no assumption about how commonly the fragment occurs. Its scope is nonetheless specific: the family also belongs to OWL 2 EL, and the relevant conclusion can be reached without explicitly constructing the unnamed individuals. The experiment does not establish superiority over specialized EL reasoners or performance on arbitrary existential ontologies.

5 Discussion

The findings support a conditional account of efficient ontology reasoning. Restricting a language can create opportunities for simpler computation, but the benefit also depends on the arrangement of facts, the available justifications, and the questions being asked. In the present experiments, sharing surviving justifications was valuable when many individuals could reuse the same reasoning. It became a cost when the attempted justification did not exist. Similarly, spending more effort choosing an evaluation order helped one demanding query, but imposed overhead on adverse cases. These results identify conditions for improvement rather than a universally preferable strategy.

The external comparisons make this distinction especially important. One rule insertion was approximately six times faster than ZodiacEdge, whereas one rule deletion was approximately forty-one times slower. Neither result alone characterizes the relative quality of the two approaches. Together, they show why an average across unlike operations can conceal a substantial weakness. They also motivate a more discriminating question: which observable properties of a change predict whether shared reasoning, selective reconsideration, or broader recomputation will be most effective? The current evidence does not yet establish a reliable general answer.

The Bach update discrepancy adds a distinct qualification. Successful recomputation of a state does not establish that every path of changes leading to it will preserve the same conclusions. The surviving family connection supplies a small, independently understandable test of this requirement. An observed failure in one recorded version warrants a reproducible counterexample and a scoped conclusion, rather than a general ranking of reasoning methods.

5.1 The continuing value of restricted languages

A fragment’s scientific value does not depend on knowing how frequently it is used in practice. A proven reduction in computational difficulty is valuable even without an adoption survey. Prior work on description logics whose statements take a restricted implication form, known as Horn description logics, illustrates this point: restricting the form of logical statements changes the established complexity of specified reasoning tasks [14]. That result concerns precisely defined languages and tasks. It does not, by itself, supply a complexity guarantee for the DLP L3 fragment or the reasoning procedure studied here.

Measured performance provides a different kind of evidence. On the earlier constructed existential family, the specialized reasoner checked consistency and identified the individuals satisfying the same questions faster than HerMiT, which supports a broader OWL language [6]. This demonstrates a useful computational specialization under the experimental conditions. It does not prove that all DLP L3 tasks are easier, that the advantage grows indefinitely with input size, or that restricting the language is the sole cause of the observed difference. Much of the advantage was already present before the new optimizations.

That constructed family also belongs to OWL 2 EL, another established restricted language. This qualification concerns the constructed experiment; the full Bach ontology has a separate semantic scope. The result therefore leaves open how the approach compares with reasoners specialized for EL. Moreover, although the experimental statements require related individuals to exist, a reasoner can derive the relevant class memberships without explicitly constructing those individuals. The experiment establishes an advantage for the specified questions; it does not establish an advantage of generating unnamed individuals.

The relationship between the examined fragments and OWL 2 RL is one of overlap, rather than a simple hierarchy. DLP L3 permits general statements requiring the existence of a related individual, which lie outside RL’s permitted syntax. Conversely, RL includes constructs absent from the reconstruction [18]. It would therefore be misleading to describe DLP L3 as a superset of RL or all the historical fragments as RL subsets. Language membership, the meaning assigned to statements, and the questions a procedure can answer completely must be assessed separately.

5.2 Limits of the evidence and further questions

The benchmark collection combines controlled examples, a university ontology, the thesis’s original Bach examples, changes to a published rule program, and selected database query components. This diversity tests several mechanisms, but does not constitute a representative sample of ontology applications. The database components omit parts of the full published workloads. Repeated measurements on one machine provide local evidence, with limited information about variation across hardware, systems, or larger datasets. Historical published timings provide context, not directly comparable speed measurements. The Bach examples retain their original small scale; repeating their measurements does not turn them into a scalability study.

Agreement with independently justified answers strengthens confidence in the reported experiments. It cannot establish correctness for every ontology, nor certify the thesis’s historical software. Likewise, corrections to particular printed

formulas and procedures do not invalidate the underlying connection between ontology semantics and rule evaluation. They indicate where the specification needs repair and where experimental reconstruction can contribute to scientific scrutiny.

Further studies can test three propositions. First, the benefit of shared justifications should increase with the proportion of individuals that share both starting facts and surviving proofs. Second, selective rule-maintenance methods should be most advantageous when a change affects a small part of the reasoning dependencies. Third, choosing an evaluation order should help only when avoided combinations compensate for the cost of making that choice. Experiments that vary these properties independently would provide stronger causal evidence than adding more undifferentiated benchmarks.

Broader comparisons should also include specialized EL reasoners and diverse DLP L3 tasks, with explicit termination and completeness conditions. Some recursive existential descriptions can make the present procedure generate an unending sequence of unnamed individuals. Failure of this procedure to finish does not establish that no other procedure could decide the same question. A separate study could examine how often real ontologies and their application questions fit each fragment. Such a survey would inform practical adoption, but its difficulty does not postpone the theoretical or computational case for studying restricted languages.

6 Conclusion

Description Logic Programs offer useful computational specializations, with advantages that depend on the knowledge, the change, and the question. Sharing valid explanations reduced selected update costs, while unsuccessful searches added work. Changing evaluation order reduced some intermediate results without consistently improving complete task times. The DLP L3 comparisons with HermiT demonstrate benefits on defined tasks, while the mixed ZodiacEdge results show why performance must be assessed alongside preservation of every required conclusion.

These findings support restricted reasoning languages without establishing a general performance ranking or a new complexity bound. Their computational value is distinct from their prevalence in applications. The next step is to test whether observable properties of knowledge and updates predict the reported benefits across broader, independently selected workloads.

Technical appendices

A Original work, semantics, and reconstruction

A.1 What is inherited from the thesis

The thesis provides the DLP-oriented problem decomposition, position-sensitive constructor translation, instance and schema reasoning services, and maintenance under changes to a knowledge base. Chapters 5–7 are the main implementation reference; chapter 8 supplies taxonomy, restriction, and update workload designs. The 2003 materialized-ontology paper already discusses changing both extensional facts and intensional rules [32].

The reconstruction compiles RDF-encoded OWL expressions into immutable Horn rules. RDFSLib supplies RDF parsing; it does not establish the correctness of the OWL-to-Horn translation. The engine uses relation indexes, semi-naïve deltas, explicit equality with congruence, integrity constraints, and a distinction between assertions and derived facts. It supports a documented DLP fragment, not full OWL DL, OWL Full, or complete OWL 2 profile conformance.

The local DLP L0–L3 names follow constructor-position distinctions in the reconstruction. For example, an existential antecedent $\exists R.C \sqsubseteq B$ is a Horn pattern match, whereas an existential consequent $A \sqsubseteq \exists R.C$ requires the witness-generating DLP L3 mode. General disjunctive heads, property chains, keys, arbitrary datatype restrictions, and maximum cardinality above one are outside the implemented contract. Automatic literal value identity covers only stated string, numeric, and Boolean families. Unsupported logical constructs fail explicitly.

Negative answers require a complete consistent computation; failure to derive a fact is not derivation of its negation. Different IRIs are not presumed to denote different individuals. DLP L3 uses Skolem witnesses; unrestricted evaluation can fail to terminate on recursive existential ontologies, while configured resource limits return an incomplete result. Retractions involving equality reconstruction or existential witnesses use rematerialization. The certificate argument below does not extend to those cases.

A.2 Focused corrections and their significance

The audit uses the thesis’s printed page numbers; the one-based PDF page is 16 higher. Table 2 gives representative corrections. The repository’s longer errata includes the countermodels, conditional benchmark-count corrections, and independent executable checks. These are findings from this implementation review; they do not establish how the historical software behaved.

Table 2: Representative audited issues in the original thesis [31]. The corrections concern the printed specification; historical software behavior cannot be inferred without its implementation and raw data.

Location	Issue	Required interpretation or replacement
Table 5.3, p. 122	Subproperty and transitivity implications point in the wrong direction.	$P(x, y) \rightarrow Q(x, y)$ for $P \sqsubseteq Q$; $P(x, z) \wedge P(z, y) \rightarrow P(x, y)$ for transitivity.
§5.4.4.2, p. 136	Joint class-equivalence probes can interfere through equality.	Run the two subsumption probes in separate fresh states.
Eq. (5.13), p. 140	Exact inverse equality is inherited to a strict subproperty.	Inherit inverse inclusion, or require property equivalence for exact inverse equality.
Algorithm 6.1, p. 163	A removed rule can retain an obsolete rederivation contribution.	Identify losses using old rules; restore only with current rules.
§5.4.2.2, p. 133	A stated distribution is false; explicit DNF need not be linear.	Correct the identity and retain factored structure or auxiliary predicates.
Tables 8.9–8.10	Restriction columns and some population labels conflict with the generator description.	Validate generated counts; do not relabel unavailable historical measurements as reconstructed data.

Two examples show why the procedural findings matter. If $C \sqsubseteq \{o\}$ and $D \sqsubseteq \{o\}$, a joint probe asserting $C(a)$ and $D(b)$ identifies $a = o = b$ and obtains both cross-memberships. Yet $C = \{o\}$, $D = \emptyset$ is a countermodel to $C \sqsubseteq D$. Fresh names alone do not isolate tests under equality. For deletion, let $A(a)$ be explicit and let $P(x) \leftarrow A(x)$ and $P(x) \leftarrow B(x)$ be the rules, with B empty. Removing the first rule must remove $P(a)$. An obsolete rederivation rewrite can incorrectly preserve it.

These findings invalidate particular printed formulas or procedures, not the entire research programme. The model-theoretic connection between the supported languages and the usefulness of rule-based ontology evaluation do not

depend on the faulty examples. Conversely, successful modern tests cannot certify the historical implementation or recover which datasets produced ambiguous old timing entries. The audit also corrects its own earlier interpretation: loose restriction scope in Example 5.4.2 is reported as ambiguous parentheses rather than a separately proven semantic error. OWL’s original direct semantics provides an independent reference for the relevant constructors [26].

B Delta-sensitive unary evaluation

For a recognized rule body $P_1(x), \dots, P_k(x)$, let I_i be its current relation and D_i the frontier for that relation. Duplicate predicates are removed. The engine ingests the frontier before evaluation, so $D_i \subseteq I_i$. The coalesced semi-naive result is

$$J = \left(\bigcap_{i=1}^k I_i \right) \cap \left(\bigcup_{i=1}^k D_i \right). \quad (1)$$

The fixed plan first constructs the delta union and intersects the necessary full relations. This can create a large temporary set even when a small full relation already bounds every possible answer.

The selector rejects an empty required relation and handles an already bound variable by exact membership tests. If a nonempty delta has the same cardinality as its full relation, the subset invariant proves equality: every full-join answer meets the delta condition. With a single nonempty delta, the engine uses that set directly. Otherwise define $M = \sum_i |D_i|$, $s = \min_i |I_i|$, and let $h \leq k$ count the nonempty deltas. The alternative branch is selected when $M > ks$:

$$T = \bigcap_i I_i, \quad J = \bigcup_i (T \cap D_i). \quad (2)$$

Set distributivity establishes equivalence with Eq. (1). Full relations are considered in cardinality order, and delta filtering may stop when its accumulated answer covers T . The selector uses current exact sizes; it does not infer independence or estimate overlap from samples.

Under expected constant-time hashing, the full-first branch uses $O(k \log k + ks + hs)$ work and $O(s)$ temporary tuple storage, excluding references and emitted bindings. A simple scan accounting charges at most ks for copying/intersecting the full relations and $2hs$ for filtering and collecting outputs. Since the branch is chosen only when $M > ks$,

$$(k + 2h)s \leq ks < 3M. \quad (3)$$

This is a scoped scan bound, not a three-competitive runtime theorem. Sorting, allocation, hash collisions, Python dispatch, and unequal operation constants are outside that inequality. Engine counters also omit work performed inside C-level set operations.

An initial rule selected full-first at $M \geq s$. A pilot falsified it: 64 identical singleton deltas over 32-row full relations have a one-row union but trigger many full intersections. The observed kernel slowdown was about 2.7-fold. The width-aware threshold was adopted before the final experiment, and this adversary was added to its protocol. The pilot is development evidence; the retained confirmatory samples concern the revised selector. Equality reindexing and DRed variants preserve their applicable delta contracts; the coalesced shortcut is not imposed on unrelated arbitrary delta relations.

C Bounded current-proof certificates

C.1 State and oracle

Let P, E be the old rules and explicit facts and let D contain the old active domain’s generated TOP facts, including the private nonempty-domain representative. Let P', E', D' describe the complete proposed transaction state. Write

$$\mathcal{M} = \text{lfp}(P, E \cup D), \quad \mathcal{M}' = \text{lfp}(P', E' \cup D'). \quad (4)$$

The incremental deletion setting is finite, positive, function-free, and equality-free, with a complete old materialization. Denials are checked separately. Existing explicit current facts already stop unnecessary deletion.

The oracle indexes only current rules of the form $H(x) \leftarrow B_1(x), \dots, B_m(x)$, with $m \geq 1$ and exactly the same variable in every atom. It excludes constants, binary atoms, multiple variables, existential terms, and constraint heads. For subject a , the seed signature contains predicates explicitly asserted of a in E' , plus TOP precisely when a belongs to the current domain. Forward propositional Horn closure of the signature gives a set of proved predicates.

Subjects with identical signatures share this computation. A cycle cannot certify itself because old derived predicates are never seeds. Newly inserted rules and assertions may supply valid replacement support: the oracle examines the complete current candidate, not only the pre-transaction program. A positive answer protects an old fact considered for deletion. Other oracle conclusions are not inserted into the store. Ordinary evaluation must still create and propagate new intermediate facts and consequences of newly inserted rules.

C.2 Preservation argument

Proposition 1 (Sound current-proof protection). *Under the stated finite positive assumptions, DRed may additionally protect queried old facts $C \subseteq \mathcal{M}'$ during overdeletion through the old program, provided restoration evaluates current definitions of deleted predicates, evaluates all inserted rules, and propagates new and restored facts to a fixed point.*

Proof. Every old fact has a finite derivation tree. When an old proof uses a removed assertion, removed rule, or departed domain seed, deletion is seeded there and propagates through that proof until it reaches a protected fact. A protected fact has a finite current proof; substituting that proof makes the cut safe. Current explicit and domain facts are safe leaves. Hence an unsupported old fact cannot survive merely through cyclic old dependencies. Restoration uses only current rules, so every restored consequence is sound.

For completeness, consider a finite current derivation. If all premises of a rule survived but its old head was deleted, evaluating current definitions of deleted predicates recovers that head. An inserted or restored premise triggers ordinary delta propagation, and each inserted rule is evaluated. An unchanged rule with all premises already in the complete old model cannot yield a genuinely new head absent from that model. Induction over current derivation height therefore recovers every current consequence. $\square$

The unary oracle supplies the required subset by induction over its rule firings. Every seed is current, and every added predicate follows from a current rule and proved premises. Even an incomplete monotone exploration is sound. This is a preservation argument for the implementation’s interface, not a machine-checked proof or a claim that the general principle is new.

C.3 Budgets and expected failure modes

The implementation allows at most 128 cached signatures and 100,000 dependency visits per transaction. Exhaustion yields “unknown” and ordinary DRed continues. These bounds restrict search, not total transaction time or memory: index construction scans current rules and assertions, each query constructs or retrieves a signature, and a context may contain many predicates. Search order can change which contexts benefit, but never the soundness of a positive answer.

Shared signatures with independent surviving support should amortize the oracle cost. Unsupported cycles provide no certificates and expose pure overhead. Unique signatures stress the cache cap. Support that depends on binary rules is intentionally outside the oracle, even when general DRed can recover it. Thus a correct oracle can still be a poor default. Learning or hand-tuning a dispatcher on these same cases would require a new held-out evaluation.

D Experimental protocol and validation

The preserved baseline is commit b1254c441731ca4fdf32ea83570ff99aaa84ac2a. Its original result file and manifest are immutable. Commit 8ce254f contains the preceding modernization; commit 5531be4 records the research implementation and evidence. The factorial experiment was run against a source snapshot whose hashes and archive are preserved. Its four variants share access-path specialization and differ only in fixed/adaptive unary planning and ordinary/certificate-guided DRed. The fixed variant is not an archived checkout of the historical baseline.

The final v3 protocol defines ten unary cases: five shapes at widths 8 and 64. Each uses 2,048 subjects except the overlapping-singleton adversary with 32. Eight support cases combine surviving alternate support, unsupported cycles, mixed new support, and unique signatures with depth parameters 4 and 32 and 256 subjects. Depth d generates predicates C_0 through C_d plus an answer predicate. Nine process blocks and four configurations yield 648 timed workers. Seeded Latin-square rotations balance position over groups of four blocks; the ninth is not fully balanced. No warm-up samples are discarded.

The primary endpoint is outer wall time of the complete `Engine.update`, including candidate validation, planning, indexing, deletion, and restoration. Input generation, hashing, and outcome checks are outside that interval. Per-case ratios are medians of the nine within-block ratios. Their paired bootstrap intervals are descriptive; nine blocks on

one host do not establish population-wide significance. Earlier v1/v2 protocols and the failed pilot are disclosed. The protocol was saved before the experiment, but was not independently timestamped public preregistration.

Every worker checks its full result against fresh materialization and records canonical input, rule, assertion, and actual closure hashes. Thirty-six small controls additionally use an independently written finite-grounding evaluator. The expected transaction digests in the primary artifact should not be confused with direct inspection of internal assertion storage; randomized integration tests separately compare that storage with independently maintained inputs. Seventy-two separate processes measure additional Python allocations with `tracemalloc`. Instrumented times never enter the timing summaries, and these allocations exclude the pre-existing store and untraced native memory.

The host is an Apple M4 Max with 16 reported CPU cores, macOS 26.5.1 arm64, Python 3.12.9, and RDFLib 7.6.0. Exclusive host use was not established: an unrelated Bach report was rewritten during the primary timing window. Its exact CPU overlap is unknown. All original samples are retained. A post-hoc support replication used new seeds and 144 fresh workers, after the production suite finished, to check the main conditional gain and adverse overhead.

The predeclared engineering screen required adaptive execution to have a geometric mean update ratio at most one and no case above 10% slowdown. Certificates required consistent intended benefits and adverse overhead at most 10%. Thresholds were not adjusted after the outcomes. The completed research validation had 593 tests, including 1,458 exhaustive small relation/delta combinations and 1,000 randomized mixed transaction states, plus independent thesis-counterexample scripts, lint, and a package build. These validate the stated implementation scope, not all OWL semantics.

E Results

E.1 Adaptive evaluation: work savings without an end-to-end gain

Table 3: Complete-update unary results (nine paired process blocks). Milliseconds; ratio below one favors adaptive planning. Intervals are descriptive.

Shape	Width	Fixed	Adaptive	Ratio	95% interval
sparse-single	8	10.623	10.628	1.0026	[0.9768, 1.0123]
sparse-single	64	100.506	99.957	1.0060	[0.9624, 1.0209]
sparse-disjoint	8	10.502	10.782	0.9961	[0.9340, 1.0409]
sparse-disjoint	64	103.556	103.567	0.9972	[0.9734, 1.0473]
sparse-overlap	8	0.271	0.269	1.0246	[0.9672, 1.1895]
sparse-overlap	64	2.132	2.147	1.0018	[0.9945, 1.0201]
dense-overlap	8	53.179	53.651	1.0019	[0.9944, 1.0327]
dense-overlap	64	688.126	697.530	1.0127	[0.9814, 1.0303]
dense-selective	8	42.324	41.733	0.9903	[0.9777, 1.0194]
dense-selective	64	656.392	667.461	0.9818	[0.9378, 1.1023]

The geometric mean of the ten paired ratios is 1.001446, with worst case 1.024637; every descriptive interval includes one. The candidate therefore fails its at-most-one geometric-mean adoption requirement. This does not prove that adaptation is generally harmful, but it supplies no basis for enabling the feature by default.

At width 64 in the dense-selective case, delta-union input visits fall from 64,512 to zero, while 32 delta-filter visits produce the same 32 body matches. Additional traced peak allocation falls only from 79,735,524 to 79,679,418 bytes. The complete transaction includes large assertion batches and index updates, so a favorable kernel transformation need not change user-visible latency. For sparse-single updates, the median fraction outside the internal evaluator timer is 99.46% at width 8 and 99.88% at width 64. This diagnostic suggests transaction preparation as a next target, but does not isolate its individual causes or prove that all of that time can be removed.

E.2 Certificates: conditional gains and adverse overhead

At supported-cycle depth 32, primary median update times change from 60.161 to 8.121 ms. The median paired ratio is approximately 0.1343, a 7.45-fold speedup; the reciprocal of separately reported medians need not equal that statistic. Overdeleted facts fall from 8,704 to zero and reported candidate rows from 17,920 to 256. One shared signature takes 35 dependency visits. Additional traced peak allocation falls from 6,418,467 to 3,029,976 bytes, or 52.8%. This is update allocation, not a total-process memory reduction.

Table 4: Certificate update ratios in the primary study and independent post-hoc replication. Each uses nine paired blocks per case.

Shape	Depth	Primary	Replay	Replay 95% interval
alternate-support	4	0.2977	0.3031	[0.2992, 0.3085]
alternate-support	32	0.1343	0.1323	[0.1266, 0.1338]
unsupported-cycle	4	1.1896	1.1784	[1.1364, 1.1947]
unsupported-cycle	32	1.1859	1.1879	[1.1537, 1.2280]
mixed-new-support	4	0.3730	0.3819	[0.3658, 0.3948]
mixed-new-support	32	0.1478	0.1495	[0.1477, 0.1532]
unique-signatures	4	0.9062	0.9053	[0.9000, 0.9170]
unique-signatures	32	0.7907	0.7849	[0.7724, 0.8319]

On unsupported cycles, certificate checks cannot prove surviving entry support. The cycles are correctly removed, but the additional work causes paired overhead of 18.96% and 18.59%. Both exceed the 10% screen. Unique signatures exhaust the context cap: at depth 32, half the 256 contexts are protected while 4,352 facts are still overdeleted. The resulting ratio is approximately 0.7907. At the shallow depth, partial time savings coexist with increased allocation.

The separate support replication preserves the finding: depth-32 alternate support has ratio 0.132315, or 7.56-fold speedup, while unsupported cycles have ratios 1.1784 and 1.1879. All 144 workers match the original case outcome hashes and fresh reference closures. This post-hoc study does not replace the primary observations or convert a workload-specific result into a general claim.

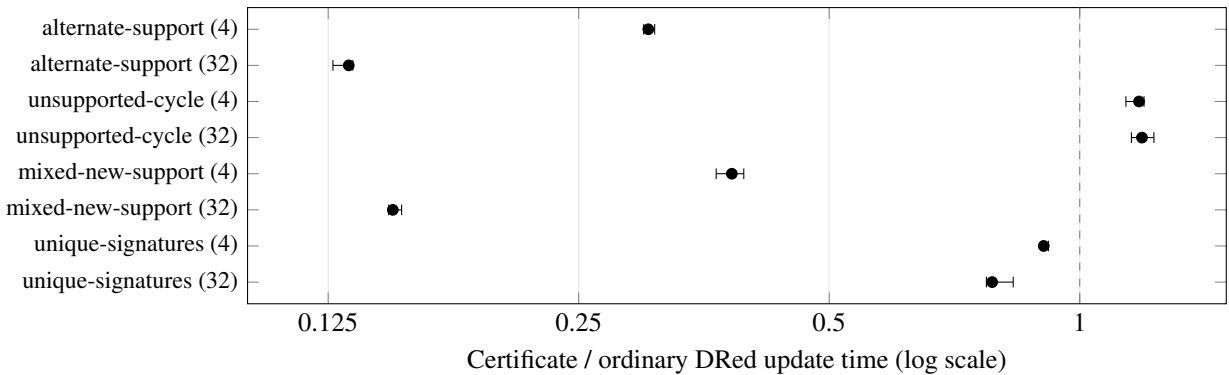


Figure 2: Post-hoc support replication. Points are median paired ratios; bars are descriptive paired-bootstrap intervals. Values left of one favor certificates; unsupported cycles expose their overhead.

E.3 Earlier production comparison with the fixed local baseline

At commit 5531be4, production kept union-first unary planning and ordinary DRed; only the tested unary/binary lookup specialization was enabled from that research phase. Its thesis suite contains 54 cases with five observations each. All 270 runs pass their checks; 51 cases match the preserved baseline’s input hashes, and three Bach additions are outside it. Table 5 reports selected longitudinal comparisons, including costs. The full artifacts retain all results, so this selection is not a summary of aggregate superiority.

Large cumulative gains on factored-union workloads and schema queries include earlier modernization. They cannot be attributed to the experimental certificates or adaptive selector, which are disabled. Sequential historical runs also differ in environmental noise. The worst recorded materialization regression against b1254c4 is approximately 13.46% on taxonomy D7/I9/P0. Neither this single number nor an isolated large improvement substitutes for matched competitive external evaluation.

Table 5: Selected historical medians (ms), five samples each. These are sequential local runs, not paired external-system comparisons. Materialization unless another operation is named.

Workload/operation	b1254c4	8ce254f	5531be4
Factored unions, width 64	10,406.030	119.371	105.388
Enumerations, width 64	95.863	6.059	5.717
Taxonomy D7/I15/PF	4,860.310	4,388.391	4,777.663
Same taxonomy: subsumption	7,983.876	12.339	12.843
Transitive, 100	451.250	710.512	446.153
Cardinality D5/I3	243.571	367.386	227.536
D5/10% rule insert	219.359	372.468	228.417
D5/10% rule replace	417.815	725.576	457.058
D5/10% rule delete	685.817	598.605	413.883
D5/10% atomic mixed	771.007	709.321	791.637

F Established benchmarks and external evaluation

F.1 Benchmark selection and comparison contracts

The Lehigh University Benchmark (LUBM) [8] is the first external workload because its maintainers supply an ontology, deterministic generator, queries, and extensional reference answers [16]. UOBM [17] increases expressive and structural demands; OWL2Bench [28] provides explicit OWL 2 profile coverage. ORE [25] supplies real-ontology reasoning tasks. WatDiv [1] and BSBM [4] instead stress RDF query processing. Table 6 identifies the relevant contracts rather than treating these suites as interchangeable rankings.

For relational workloads, the Transaction Processing Performance Council’s Benchmark H (TPC-H) evaluates analytical queries over business data [30]. Our use of selected query components is described below.

Table 6: Established benchmark candidates and their applicability. Selection is our assessment; benchmark purpose and language come from the cited sources.

Benchmark	Appropriate use	Required boundary
LUBM	Loading, materialization, exact named query answers	Official ontology and generator; DLP L3 for the unmodified ontology here.
UOBM	More connected and expressive ontology workloads	Pin the generator: Oxford’s version changes distributions; reject unsupported logical features.
Oxford maintenance suite	Compare DRed, DRed ^c , B/F, B/F ^c	Match the exact transformed rule program and fact-deletion batches.
ORE 2015	Coverage, consistency, classification, realization	Record unsupported ontologies; a filtered DLP track cannot inherit full DL/EL rankings.
OWL2Bench	Feature coverage and scaling by OWL 2 profile	Property chains and some other profile features exceed this compiler’s contract.
WatDiv / BSBM	Join diversity and RDF query services	Separate query adaptation from inference; not tests of DRed or OWL completeness.

F.2 Official LUBM(1,0): independent answer validation

The initial external study uses UBA 1.7’s unchanged bundled generator bytecode, university index zero, seed zero, and one university. Fifteen department files are generated. The original ontology IRI is retained; imports are resolved locally and their 15 document import statements removed after resolution. No logical ontology axiom is projected away. The resulting graph has 100,851 distinct triples, including 293 ontology triples. L2 rejects existential consequents; DLP L3 accepts the original ontology and completes materialization.

The official query file contains punctuation errors and an older namespace. The adapter records both the upstream bytes and its normalized queries, fixing only those syntax/namespace differences. RDFLib parses each basic graph pattern; the reasoner’s existing indexed join evaluator executes it. This is an explicit BGP adapter, not a complete SPARQL frontend. Bindings include named individuals and literal values, matching the official extensional answers. Skolem witnesses remain present in the materialization and in intermediate reasoning.

All nine fresh workers (three configurations, three balanced blocks) return exactly the official answer tuples for all fourteen queries: 126 successful query checks, including both missing and excess tuple detection. In query order, the cardinalities are

$$(4, 0, 6, 34, 719, 7790, 67, 7790, 208, 4, 224, 15, 1, 5916).$$

The full internal closure contains 234,480 facts; its named Univ-Bench projection contains 138,478. All nine full internal closure hashes also agree. Internal and named closure hashes are recorded separately; agreement does not independently certify every unqueried existential consequence. Reference-answer equality is the stronger external evidence for the named benchmark questions.

Table 7: Official LUBM(1,0), three fresh processes per configuration. Loading/reasoning times are seconds; the fully consumed fourteen-query total is milliseconds. Earlier current state is commit 5531be4.

Configuration	Parse + reasoner	Compile	Materialize	Queries (ms)
b1254c4	7.423	1.379	3.322	217.714
current-fixed	7.218	1.362	3.180	151.097
current-adaptive	7.387	1.395	3.310	150.822

Here “current” denotes the earlier 5531be4 research state, before the subsequent optimizations motivated by the extended literature review. It is preserved as a separate external control. The fixed variant has paired median ratios of 0.957 for materialization and 0.6925 for the sum of fourteen query times against b1254c4. This is a local same-host comparison, not a comparison with the thesis’s historical software. Three blocks and one university establish neither robust scalability nor an external performance frontier.

Query preparation is outside the query timer, and each query is fully consumed once per worker. Cold denotes a fresh Python process, not a flushed operating system filesystem cache. Compilation and materialization are subintervals of reasoner construction; they must not be added to the already inclusive parse-plus-reasoner column. Peak RSS includes RDF export and validation and is approximately 984 MiB. The certificate mechanism concerns retractions and is inactive in this static benchmark.

F.3 Published reference results and what they establish

The 2005 LUBM study [8], Table 1 (author PDF p. 15), reports LUBM(1,0) loading times of 13 s for Sesame-Memory, 542 s for Sesame-DB, and 343 s for DLDB-OWL. Table 3 (author PDF p. 31) gives Q1 latencies of 15, 46, and 59 ms respectively, all with four answers. Its 103,397-triple ABox count includes duplicates. The machine was a 1.8 GHz Pentium 4 with 256 MB RAM, Windows XP, Java 1.4.1, and a 512 MB heap; Sesame 1.0 used RDFS inference and DLDB-OWL used its 2004-03-29 release. These loading, storage, and semantic configurations differ from our DLP L3 materialization and BGP adapter. The numbers are historical reference points and must not be divided by our times.

For the new maintenance hypothesis, the Oxford workloads are more relevant than a query-only benchmark. Table 8 records two verified rows from Hu et al. [11], Section 6 and Table 1 (author PDF pp. 7–8). That study averages ten randomly selected 1,000-fact deletions on a Dell PowerEdge R720 with two 2.6 GHz Xeon E5-2670 processors, 256 GB RAM, and Fedora 24. It demonstrates why plain DRed alone is an insufficient competitive control for a certificate optimization.

Table 8: Published small-deletion reference times from Hu et al. [11]. Seconds, on their hardware and transformed programs; these are not new runs.

Program	Explicit facts	DRed	DRed ^c	B/F	B/F ^c
Reactome-U	12.5 million	1.03	0.06	1.00	0.05
UOBM-U	254.8 million	1,185.87	179.24	31.96	1.18

The U transformation is complete but unsound relative to the original OWL ontology; R is sound but incomplete, and Claros-LE adds manually chosen rules. These labels describe OWL approximation, not an error in Datalog evaluation of the transformed program. The original counting artifact server could not be retrieved during this study. No version-identical rerun of those Oxford data or external DRed^c/B/F^c executable is claimed.

Other widely used benchmarks expose different limitations. ORE evaluates consistency, classification, and realization under DL/EL task limits; filtering to supported DLP inputs changes the competition population. OWL2Bench includes features such as property chains that this compiler currently rejects. WatDiv uses diverse SPARQL patterns and BSBM

includes service-level and aggregate query workloads. Successful Horn closure does not implement those complete interfaces. Their proper role is a separate coverage or query track, with unsupported features and conversion costs recorded.

The final published RPT+ evaluation by Qiao, Boncz, and Zhang [27] uses DuckDB 1.3.0 and eight threads on two Xeon Platinum 8474C processors with 512 GB DDR5 memory under Debian 12. Its benchmark population includes 18 TPC-H join queries at SF100, 113 Join Order Benchmark queries, eight Appian queries, and 18,251 SQLStorm queries (13,308 completed, 2,837 failed, and 2,106 reached a ten-second timeout). Reported geometric-mean speedups are 1.17 for TPC-H, 1.47 for JOB, 1.01 for Appian, and 1.28 for SQLStorm. The final PVLDB paper is the reference here; an earlier author-hosted draft has different numbers. Our smaller positive join components do not measure those full SQL workloads, and their local speed ratios cannot be ranked against these values.

The pinned ZodiacEdge repository [34] reports LUBM(1) initial, rule-insertion, and rule-deletion times of 588.9, 216.9, and 252.5 ms for the $*$ subset, and 603.7, 56.3, and 21.5 ms for the $\#$ subset. These are author README observations, separate from the paper’s wind-turbine workloads with aggregation and negation [33]. Hardware, generation seed, repetition count, and exact timing boundaries are not supplied for these README rows. The supplied positive LUBM program nevertheless permits a stronger test: run the author’s unmodified engine and our implementation on one host with the same facts, rules, update sequence, and validated semantic outputs.

G Follow-up implementation informed by the literature

The literature review and the first experiment changed the optimization target. The earlier selector saved inner-evaluator work while transaction preparation dominated several complete updates. Changing-rule maintenance already has a substantial history [15, 32, 33]; recent join research emphasizes the costs of execution choices and filtering [27]. We therefore implemented three further specializations and evaluated them against both preserved commits and a same-source control. These are engineering adaptations of established ideas. They neither implement RPT+ nor replace B/F^c, modular maintenance, or a worst-case-optimal join.

G.1 Reusing validation for monotone transactions

The engine records the validated rule sequence, assertion set, and predicate arities. An insertion-only transaction may reuse this certificate only when the public rule and assertion containers still match it exactly. New facts and rules are checked against the recorded arities and one another before the transaction changes state. Submitted removals, a missing certificate, or direct container mutation trigger complete validation. Deletion can release the last occurrence fixing a predicate’s arity, so applying the monotone shortcut there would be unsound.

Proposition 2. *For an unchanged validated input state whose embedded terms and identifiers have stable values, equality, and hashes, checking the added facts and rules against its arity map preserves the engine’s input-validation conditions for insertion-only transactions.*

Proof. Groundness and permitted-term conditions on old facts, and rule safety and permitted-term conditions on old rules, continue to hold because those objects retain their structure and embedded values. These local conditions are checked on every genuinely new object. The remaining shared condition is that every occurrence of a predicate has the same arity. Extending a copy of the old arity map while checking all new occurrences enforces that condition across old and new objects and within the additions. The extended certificate is installed only after these checks succeed. $\square$

The API’s frozen rule and atom records are shallow: arbitrary custom hashable values can still have mutable internals. The stated stable-value assumption is required for both the certificate and ordinary hash indexes. RDF terms and the primitive identifiers used in these experiments satisfy it; snapshot equality does not detect arbitrary in-place changes inside custom value objects.

When no rule is genuinely new, prepared plans are retained. A complete old materialization has already registered its domain, including the private nonempty-domain seed. Insertion of new rules therefore requires explicit domain seeding only for their previously unseen constants; ingesting new facts registers their own terms. Incomplete old states retain rematerialization. Snapshot equality, assertion-set construction, and immutable snapshots still incur work proportional to the existing state. This is a reduction in repeated Python validation and preparation, not an $O(|\Delta|)$ bound for the complete transaction.

G.2 Maintaining surviving DRed indexes

After overdeletion has finished using the old closure, ordinary DRed rebuilds the index over the survivors. The candidate can instead erase each overdeleted tuple from its relation and every corresponding column bucket. It uses erasure when $2|D| \leq |I \setminus D|$, where D is the overdeleted set and I the old indexed closure, and otherwise rebuilds. Empty buckets and relations are removed. The threshold is an implementation heuristic, not a competitive runtime guarantee.

For each relation P , column j , and value v , the index invariant is

$$B_{P,j,v} = \{t \in I_P : t_j = v\}.$$

Erasing every $d \in D_P$ from I_P and from B_{P,j,d_j} leaves exactly the buckets obtained by rebuilding over $I_P \setminus D_P$. Deletion occurs only after the overdelete traversal, so its old-index access remains valid. Rederivation then sees the same survivor index under either implementation. The eligibility checks for equality, function terms, and complete old closures are unchanged. This optimization reduces index construction costs; it does not reduce the logical overdeletion set or solve incremental equality splitting.

G.3 Positional bindings for ordinary positive joins

For positive bodies without equality or inequality atoms, a reusable plan assigns each variable a position in a binding vector. Atom terms are compiled to variable positions or constant positions. Each recursive step retains the existing dynamic choice of the smallest available exact index bucket, extends a copied binding vector, and checks repeated variables. Constants are canonicalized on every invocation because equality representatives can change. The final vector is converted back to the original variable-binding interface. Unary same-variable plans and the generic fallback remain available.

For validated bodies, the preservation argument is an induction on the remaining body atoms; nonground Skolem patterns in bodies are rejected by validation. A vector position represents exactly the binding of its corresponding variable. The chosen index bucket is a superset of matches for the current partial binding; constant and repeated-variable checks retain precisely the compatible tuples. Every compatible extension recurses on the remaining atoms. At an empty body, conversion preserves all bindings. Semi-naïve evaluation keeps the designated delta *occurrence*, including when several atoms share a predicate. Changing atom order does not change the set of substitutions. This specializes an interpreter; it adds neither semijoin filtering nor the Bloom-filter pipelines of Qiao et al. [27].

Before the external timing series, the complete suite passed 698 tests. New controls include 420 randomized insertion states, 140 randomized mixed transactions with index comparisons, and 180 generated relational bodies compared with an independent Cartesian-product evaluator. Tests also cover failed-validation atomicity, public-container mutation, new domain constants, equality fallback, and repeated delta predicates. These checks address the implemented contracts rather than full OWL conformance.

G.4 Matched author-program and SQL-component protocols

The four local configurations are immutable commits b1254c4 and 5531be4, the new source with all three switches disabled, and that same source with all three enabled. The earlier adaptive unary selector and support certificates remain disabled throughout. The same-source control distinguishes the enabled bundle from changes that affect both paths. Comparing the bundle does not isolate the contribution of each maintenance optimization.

The ZodiacEdge author repository [34] supplies a 128-rule positive LUBM program and two marked rule subsets: eight rules marked $*$ and sixteen marked $\#$, with one rule shared. Each scenario materializes without the marked rules, inserts them, and removes them. Official UBA generation provides 100,543 ABox facts after excluding ontology metadata. The author’s generation seed and file hashes are unavailable, so matching its published input count is not proof of byte-identical data. The final author-program closure has 238,474 non-domain tuples, including the 100,543 wrapped input facts. It is a different semantic artifact from full-ontology DLP L3 reasoning.

Four balanced process blocks evaluate all four configurations. Ten fixed, seeded samples each remove and reinsert 1,000 assertions under the complete program. This follows the small-update scale used by Hu, Motik, and Horrocks [11], but does not reproduce their Reactome or UOBM programs. Thirteen fresh naïve reference closures cover the full program, both partial programs, and the ten fact-deletion states. Every timed worker checks complete typed-tuple digests after every transaction outside the timer. The ten samples within one worker are correlated: forty paired sample ratios are not forty independent process replicates. Reported summaries are descriptive.

The Qiao-related component experiment uses official DuckDB dbgen data at scale factors 0.01 and 0.1 and the positive joins underlying TPC-H Q3, Q5, and Q9. It retains the original tables and filters, but projects every participating row’s

complete primary key. Under the checked key constraints, that projection identifies the entire tuple of input rows injectively and therefore preserves the pre-aggregation SQL bag’s multiplicity. Exact answers are checked against independently executed SQL over the original database. The engine’s rule-free base closure is independently checked as its input facts plus all required domain facts.

DuckDB performs date/string selections and projection; their time and complete transfer to Python are recorded. Fact conversion, engine construction/indexing, and complete join consumption are separate intervals, all included in the cold component total. The three components run sequentially inside each fresh worker, with a new reasoner per component but shared DuckDB and filesystem caches. Aggregation, arithmetic, ordering, and top- k are outside this experiment. Three blocks, two scales, and four configurations yield 24 fresh workers and 72 component observations. Three rotations do not completely balance four configurations. These small components are useful regression and optimization probes, not a full TPC-H result or a reproduction of Qiao et al.’s SF100 evaluation.

G.5 Follow-up measurements

Table 9: Author-supplied ZodiacEdge LUBM program: four fresh process blocks per configuration. Median seconds; initial includes engine construction and materialization. Parsing and outcome validation are outside these intervals.

Subset	Configuration	Initial	Insert rules	Delete rules
*	b1254c4	3.5382	0.3338	2.3322
*	5531be4	3.2811	0.3343	2.1418
*	current-fixed	2.9002	0.3182	1.9661
*	candidate	2.7809	0.1020	1.3576
#	b1254c4	3.2615	0.4923	1.5962
#	5531be4	3.1646	0.4933	1.5541
#	current-fixed	2.7764	0.4403	1.4061
#	candidate	2.4551	0.2458	1.0838

Table 10: Supplemental 1,000-fact updates on the same author program. Median seconds across ten fixed subsets in each of four process blocks. These are forty correlated case/block observations per configuration.

Configuration	Delete facts	Reinsert facts
b1254c4	2.3727	0.1349
5531be4	2.1627	0.1210
current-fixed	1.8719	0.1141
candidate	1.4258	0.0395

Table 11: Fully consumed join kernel for TPC-H-derived Q3/Q5/Q9 components (ms). Three fresh workers per scale/configuration. Final column is the median paired candidate/current-fixed ratio; smaller favors the candidate.

SF	Join	b1254c4	5531be4	Fixed	Candidate	Ratio
0.01	Q3	5.03	4.76	4.69	3.69	0.755
0.01	Q5	20.26	21.30	21.26	11.34	0.538
0.01	Q9	35.98	34.76	34.43	23.08	0.670
0.1	Q3	45.10	44.80	43.05	31.78	0.752
0.1	Q5	941.87	957.38	949.31	500.09	0.523
0.1	Q9	390.87	378.17	380.53	266.40	0.700

G.6 An actual external engine on the same host

The comparator uses the pinned, unmodified ZodiacEdge Program API and parser. Its unary atoms use physical RDF type triples, whereas our store uses unary relations. An injective encoding retains RDF term identity, and both outputs are decoded to the same typed tuples for complete closure checks. Native input index creation is included in initial compute. Conversion is recorded separately and excluded from both implementations’ compute intervals. Native whole-process RSS includes adapter allocations, so it is not an isolated engine-memory ranking. The program has no predicate variables; fixed predicate labels and injectively encoded data constants occupy separate namespaces. Translating unary atoms to type triples and binary atoms to property triples therefore gives a bijection between rule substitutions. This validates the adapter’s semantic contract; full-run checks still test the engines’ actual computation.

Table 12: Cold component total for TPC-H-derived Q3/Q5/Q9 components (ms). Three fresh workers per scale/configuration. Final column is the median paired candidate/current-fixed ratio; smaller favors the candidate.

SF	Join	b1254c4	5531be4	Fixed	Candidate	Ratio
0.01	Q3	224.58	221.31	226.80	223.30	0.985
0.01	Q5	428.23	429.61	438.62	419.32	0.974
0.01	Q9	657.89	650.38	654.30	627.63	0.959
0.1	Q3	4,040.74	4,130.65	3,999.45	4,057.00	1.008
0.1	Q5	8,767.87	8,767.37	8,712.05	8,112.88	0.919
0.1	Q9	11,740.80	11,615.00	11,533.75	10,994.71	0.952

Table 13: Matched comparison with the unmodified author engine: four alternating paired blocks, seconds. Initial includes native input indexing. Final column is the median paired candidate/native ratio; below one favors our candidate.

Subset	Operation	ZodiacEdge	Candidate	Ratio
*	Initial	1.4001	2.4567	1.756
*	Insert rules	0.5590	0.0880	0.161
*	Delete rules	0.7850	1.1965	1.522
#	Initial	1.6595	2.1807	1.318
#	Insert rules	0.0589	0.2138	3.653
#	Delete rules	0.0234	0.9709	41.335

G.7 Interpreting the maintenance improvement

All 16 ZodiacEdge-program workers pass all 416 recorded complete-closure stage checks. Relative to b1254c4, median paired insertion ratios are 0.3144 for * and 0.4807 for #, corresponding to 3.18-fold and 2.08-fold speedups. Deletion ratios are 0.6119 and 0.6969. These are new matched local comparisons on the author’s program, not ratios against its published README times.

The same-source control is especially useful for attribution. Candidate/fixed rule-insertion ratios are 0.3265 and 0.5470; deletion ratios are 0.7240 and 0.8230. Every individual paired update ratio favors the candidate in both scenarios. For initial compute, however, the * median ratio is 0.9770 and two of the four individual ratios exceed one. We do not interpret that small aggregate difference as a reliable initialization benefit. The # initial ratio is 0.9051, with all four observations below one. No predeclared statistical significance test or broad default-adoption criterion was attached to this follow-up study.

Across the ten fixed 1,000-fact subsets and four blocks, pooled matched candidate/b1254c4 ratios are 0.6251 for deletion and 0.2808 for reinsertion. Against the same-source fixed control they are 0.7169 and 0.3064. These medians describe the sampled workload, with cases nested within processes. They do not supply forty independent repetitions or establish the performance of Oxford’s larger UOBM or Reactome workloads.

The candidate preserves the original logical DRed work contract. For example, in the first block it erases 24,933 indexed tuples for * rule deletion and 24,045 for #, rather than rebuilding all survivor buckets. Insertion can reuse validated old inputs and plans. The experiment tests the combined changes; it does not independently assign an exact portion of each speedup to validation, index erasure, or positional joins. The original adaptive unary strategy and proof certificates are disabled, so these results do not reverse their earlier default-adoption findings.

G.8 Positive-join gains and remaining intermediate work

All 72 TPC-H component observations pass exact SQL row-identity and complete base-closure checks. Paired candidate/fixed join ratios range from 0.523 to 0.755 over the six scale/component cases. The corresponding component totals range from 0.919 to 1.008. Large Q5 improves about 47.7% in the join interval, but only 8.1% in the sum of extraction, encoding, indexing, and joining. Large Q3 does not show a component-total gain. These differences matter because the Python loading and representation costs dominate this adapter.

Candidate and fixed configurations enumerate identical numbers of candidate rows in every case. Q5 increases from 13,739 to 621,901 candidates between SF0.01 and SF0.1, a 45.27-fold expansion for approximately tenfold input growth. Positional bindings reduce interpretation cost but leave this join order unchanged. This observation motivates the separately versioned lookahead experiment below; it does not by itself identify the causal ordering decision.

The original component worker executes three queries sequentially. Later-query fact encoding was faster in the candidate despite identical encoding logic. Inspection found recursive local generator closures retaining captured state until cyclic garbage collection. The earlier summed phase intervals exclude explicit teardown and validation, so they are not all application CPU costs. This is a plausible interaction, not a proven causal explanation of those timings. The follow-up breaks the closure cycle in both new arms and gives each component its own process, while retaining the original observations unchanged.

G.9 The external comparison is mixed

All eight native-comparison workers pass their six full-closure checks, giving 48 checked states. Relative to the unmodified ZodiacEdge engine, our candidate’s paired * insertion ratio is 0.161, approximately a 6.2-fold improvement. For * deletion, however, the ratio is 1.522; for # insertion it is 3.653, and for # deletion it is 41.335. Initial computation also favors ZodiacEdge. One native # insertion is an outlier at approximately 0.215 seconds, compared with approximately 0.058 seconds for the other three; all observations remain in the artifact. Four paired blocks justify descriptive comparisons on these operations, not a broad statistical or state-of-the-art ranking.

The large # deletion gap warrants diagnosis before another algorithmic claim. Index erasure reduces physical rebuilding but leaves transaction-wide preparation passes in place. A separately instrumented run, described below, attributes most of this case’s cost to those passes, while its overdeletion set is exactly the set of lost facts. Thus this particular gap should not be attributed to unnecessary logical overdeletion. Dependency-local maintenance and safe reuse of validation evidence are relevant next directions, informed by ZodiacEdge [33] and modular materialization [13].

G.10 Diagnostic profile of the rule-deletion gap

After the paired native comparison, we profiled one deletion of the sixteen #-marked rules using CPython 3.12.9 cProfile. The process used the frozen primary candidate, its three enabled optimizations, the same 100,543 assertions and 128-rule program, and hash seed 7300. It first materialized the partial program and inserted the marked rules; only their subsequent removal was profiled. All three full typed-tuple closure checks passed. This is a separate instrumented diagnostic, not an additional timing replicate.

The profiled public update accumulated 2.130745 seconds. Four disjoint call groups accounted for 1.553658 seconds, or 72.9%: canonicalizing all 100,543 assertions for protection, 0.551646 seconds; two checks of old and new DRed eligibility, 0.520192 seconds; full input validation, 0.295590 seconds; and recomputing active-domain constants, 0.186231 seconds. These cumulative times include descendants but not one another. Nested normalization and literal-key operations, or the enclosing DRed routine, must not be added again. All other update work accounts for the remaining 0.577087 seconds. The engine’s internal update counter excludes validation and eligibility checks and therefore covers a narrower interval.

Only sixteen rules were evaluated. The deletion overdeleted 24,045 facts and rederived none, exactly matching the difference between the complete closures of 238,474 and 214,429 tuples. Thus this observation contains no excess logical overdeletion. Global preparation and protection passes dominate the observed candidate cost. Since the native engine was not profiled, this result does not quantitatively decompose the separately measured 41.3-fold runtime ratio or show that eliminating those passes would close the gap.

Maintaining auxiliary state incrementally is established practice in view and materialization maintenance [9, 23]; dependency analysis also limits reconsideration under rule changes [15, 33]. Applying these general principles to the engine’s metadata suggests eligibility certificates, predicate occurrence counts, and active-domain reference counts. The exact proposals here are not claims about the implementations in those papers. Any certificate must detect relevant public-state changes and handle datatype aliases and equality history. Arity counts must release a predicate’s constraint when its last occurrence disappears; domain counts must handle constants supported only by removed rules and preserve the nonempty-domain seed. Within an eligible DRed update, identity representatives also suggest avoiding repeated canonicalization of unchanged assertions while preserving inequality symmetry and domain protection.

These changes remain proposals, not implemented or measured improvements. Deterministic profiling perturbs runtime, particularly for call-heavy Python paths, and this single transaction cannot establish general workload behavior or an achievable speedup. The compact artifact `benchmarks/zodiac-deletion-profile.json` records frozen source and profiler source hashes, the raw profile hash, independent closure checks, nonoverlapping costs, and the instrumented engine counters.

H A post-hoc bounded join-order experiment

The Q5 intermediate expansion prompted a second, separately frozen experiment. The candidate draws on a lesson from RPT+ and Shredded Yannakakis [3, 27]: reducing intermediate work is useful only when decision and reduction costs are included. It implements a smaller ordering heuristic, with neither their filtering architecture nor their guarantees.

H.1 Method and preservation argument

At a recursive positive-join step, the candidate considers the two smallest current index buckets only if the smaller contains at least 32 rows and the larger is at most four times its size. For each candidate bucket it inspects at most four physical rows, performs exact unification, and probes the other remaining atoms. The score is the current bucket size multiplied by one plus the average smallest next-step bucket size. A sampled row that fails repeated variable or constant checks contributes zero next-step work. Ties retain the original greedy choice. Samples are traversal estimates, never filters.

Hints are cached only for the invocation, keyed by remaining atom occurrences, bound-variable positions, the two candidate occurrences, and bucket-size bit-lengths. They contain an occurrence identifier, not answer bindings. This coarse reuse may choose a poor order for a different binding, but cannot remove an answer. The fixed limits are 1,024 additional index probes and 1,024 hints; together they bound sampled physical rows by 8,192. A separate hint limit is necessary because invalid sampled rows can consume no probes. Exhaustion uses greedy evaluation, including ignoring already cached hints. These bounds do not bound the cost of a poor chosen order or the query-width-dependent cost of copying bindings and constructing keys.

Proposition 3. *For a finite completed store and a validated positive relational query, the bounded estimator preserves the complete substitution set of the positional evaluator.*

Proof. At each step the estimator chooses one remaining atom occurrence. The evaluator then enumerates its entire selected index bucket and applies the same exact compatibility tests as before. Every compatible extension recurses on precisely the other occurrences. Induction on their number therefore yields all and only the original substitutions. A misleading sample, reused hint, or exhausted budget changes only the chosen occurrence. No estimate participates in a compatibility test or in acceptance of a completed substitution. $\square$

The implementation enables this path only for external headless queries on a complete store. Installed rules, delta variants, incomplete stores, and equality/difference builtins retain their existing evaluator. Hints are released when the query ends. Independently of ordering, both new evaluator arms now break a recursive local closure’s self-reference on exhaustion, explicit close, and exception. Tests with cyclic collection disabled check prompt release of the captured engine and index. That lifecycle change is shared by the two new arms, so their comparison isolates the ordering switch.

H.2 Protocol

Three balanced blocks compare b1254c4, the new compiled evaluator, and that evaluator with lookahead. Each TPC-H component has its own fresh process; two scales and three components therefore require 54 workers. Another 36 fresh workers evaluate near-unique, nonselective, skewed, and pairwise-consistent but globally empty cyclic controls. The same hash seed is used within a block. Sampling follows physical bucket order and is not a random statistical sample.

The unchanged original component definitions, SQL oracle, and row-identity projection are reused. The new driver and package are archived separately; the discovery study is not overwritten. Complete answer consumption, planning, and recursive-closure cleanup are included in join time. The component total adds extraction, encoding, and indexing. Complete process wall time additionally includes startup, integrity hashing, validation, serialization, and teardown; it is a reproducibility diagnostic rather than isolated query latency. Database hashing warms filesystem caches, and the operating-system cache is not flushed.

This is a discovery-workload follow-up, not held-out validation. The parameter values were fixed before its timed series; the preceding Q5 pilot is disclosed and excluded from the timing summaries. Small independent finite-grounding tests exercise constants, repeated variables/predicates, initial bindings, budget exhaustion, and the zero-probe cache-bound case. Timed adverse controls also compare with an independent nested-loop oracle. Lookahead remains experimental and disabled by default.

Table 14: Single-component fresh-process study: median join milliseconds, followed by paired lookahead/compiled time ratios for join, component total, and complete process. Three balanced blocks; ratios below one favor lookahead.

Case	b1254c4	Compiled	Lookahead	Join	Total	Process
0.01/Q3	6.499	4.436	4.628	0.996	0.986	0.979
0.01/Q5	58.258	48.740	49.326	1.001	1.016	1.011
0.01/Q9	48.721	32.406	33.060	1.040	1.004	0.998
0.1/Q3	58.692	42.424	43.047	1.015	1.001	1.003
0.1/Q5	923.572	515.218	260.271	0.510	0.975	0.990
0.1/Q9	398.469	262.076	272.030	1.043	1.000	0.998

Table 15: Synthetic adverse controls using the same protocol and columns as Table 14. All planning and complete result enumeration are charged to the join interval.

Case	b1254c4	Compiled	Lookahead	Join	Total	Process
unique	0.316	0.283	0.309	1.123	1.019	0.990
nonselective	119.297	7.757	8.272	1.066	1.029	1.006
skewed	0.399	0.282	0.341	1.155	1.027	0.995
empty-cycle	0.296	0.177	0.220	1.238	1.064	1.008

H.3 Results and adoption decision

All 90 workers pass their independent answer and full base-closure checks. At SF0.1, Q5 candidate visits fall from 621,901 to 155,969 in every block. Each lookahead invocation performs 30 extra probes, examines 16 physical rows, retains two hints, and changes 101 ordering choices. The paired lookahead/compiled join ratio is 0.509923, with observed range [0.480504, 0.519889]. Against b1254c4, which also differs in earlier indexing and interpreter improvements, the join ratio is 0.273615 [0.268051, 0.284502]. These comparisons isolate ordering only when the denominator is the same-source compiled arm.

The result does not establish a robust pipeline gain. Large-Q5 component-total paired ratios are 0.975196 [0.971553, 1.272566], and process ratios are 0.990329 [0.976037, 1.284827]. Separate component-total medians are 9.298 seconds with lookahead and 7.307 seconds compiled, despite the median paired ratio being below one. The block-specific shifts explain why the two summary statistics differ; neither should be substituted for the other. One block has a slower lookahead pipeline, largely in phases preceding query execution. All observations are retained. The experiment did not control desktop activity or processor affinity, and the preparation code is shared between these arms. The data do not establish that lookahead caused the preceding-phase variation.

The other five TPC-H components keep their candidate-row counts unchanged. Their paired join ratios range from 0.996 to 1.043, with the larger Q9 case about 4.3% slower. The four adverse-control join ratios are 1.123 for unique joins, 1.066 for nonselective joins, 1.155 for skew, and 1.238 for the empty cycle. Thus the extra decision machinery has measurable costs even when it does not change the traversal. The very large nonselective improvement against b1254c4 is already present in the compiled control and must not be attributed to lookahead.

The heuristic remains disabled by default. Its conditional Q5 success supports further investigation of intermediate-sensitive ordering, while the adverse results reject a universal benefit. No workload-specific dispatcher was tuned on these final observations. The stored protocol was frozen locally before measurement, not independently timestamped or publicly preregistered. This post-hoc discovery-workload result establishes neither RPT+/SYA superiority nor worst-case-optimal or no-regret evaluation.

I A matched existential task against a broader OWL reasoner

We compare immutable baseline b1254c4, the frozen follow-up candidate, and Hermit [6] on a finite DLP L3 entailment task. This is an original controlled family, not an established corpus benchmark. It tests computational specialization without assuming a frequency of suitable ontologies in deployed applications.

I.1 Input, task and independent semantic controls

All arms receive identical RDF/XML bytes containing

$$A \sqsubseteq \exists R.B, \quad \exists R.B \sqsubseteq C,$$

with $N \in \{100, 1000, 10000\}$ named A instances, one further named B-only individual, and an unrelated declared class D. Each worker establishes consistency and retrieves all named C and D instances. The expected C answers are exactly the N A names; D has no entailed named instances.

The GCIs entail $A \sqsubseteq C$. A model with one shared B witness reached from all A individuals, the B-only control outside C, and D empty satisfies the ontology and establishes the negative-answer obligations. An empty certain-answer set for D does not assert membership in its complement. Removing either GCI permits a model with C empty. All nine separate preflight processes—three arms on the full input and each omitted-GCI control—match these independently justified answers. Ten small Python tests also pass, including the semantic controls and rejection of a corrupted dependency jar.

Both existential axioms are essential to C entailment, but explicit witness construction is not. HerMiT uses its normal factory, classification and realization behavior; the adapter does not replace the GCIs with $A \sqsubseteq C$. We compare consistency and exact named answers, not complete materializations or generated witness identities. The input family is also in OWL 2 EL [18]; a result against HerMiT does not establish superiority over specialized EL reasoners. Although the existential consequent is outside structural RL, RDF-based subclass propagation can also answer this particular C query.

I.2 Pinned implementations and timing boundaries

The candidate comes from the hash-verified `benchmarks/followup-source.tar.gz`; it is the same package measured in the ZodiacEdge and TPC-H follow-up studies, regardless of subsequent source changes. The external distribution is the OWLAPI-maintained Maven artifact `net.sourceforge.owlapi:org.semanticweb.hermit:1.4.5.519`, with OWLAPI 5.1.9. Its POM identifies it as a fork not officially supported by the original HerMiT developers.¹ The bundle and Maven metadata identify 1.4.5.519, while the API reports 1.4.1.513: bytecode inspection confirms that the pinned jar itself constructs `Version(1,4,1,513)`. Both identifiers are retained. This comparison is not a claim about the latest upstream distribution. Eleven downloaded Maven artifacts and all 43 resulting classpath jars have recorded SHA-256 hashes.

Three blocks rotate the three arms by block and population position, giving 27 fresh processes on the same macOS 26.5.1 ARM64 host. Python is 3.12.9; OpenJDK is 17.0.18 with `-Xms64m -Xmx2g`. The heap setting is not an isolated memory measurement. All other timed studies were idle. A 180-second worker limit, source/input hashes and order were recorded before measurement. No worker failed, exceeded its limit or changed the measured source or input. All 27 workers satisfy the exact answer and consistency contract.

Task time sums loading, reasoner construction, consistency, and fully consumed C then D queries. Phase times are retained because eager DLP materialization and lazy HerMiT reasoning place work in different operations. A separate parent timer covers the complete fresh process, including runtime startup, answer serialization and exit. Input generation, dependency preparation, Java compilation and the semantic preflight are excluded. The worker series took 18.18 seconds. At $N = 10000$, HerMiT's median consistency and positive-query phases take 1.176 and 0.903 seconds; a constructor-only comparison would omit substantial work.

Table 16: Finite DLP L3 bridge task on identical RDF/XML inputs: median seconds over three fresh process blocks. Task includes loading, construction, consistency and both complete named-instance queries; process additionally includes runtime startup, answer serialization and exit.

Named A instances	Configuration	Task	Process
100	b1254c4	0.010029	0.101472
100	candidate	0.009945	0.099147
100	HerMiT	0.220450	0.272145
1,000	b1254c4	0.086836	0.188231
1,000	candidate	0.083865	0.178843
1,000	HerMiT	0.318659	0.372038
10,000	b1254c4	1.016922	1.135562
10,000	candidate	0.937498	1.055745
10,000	HerMiT	2.475294	2.532972

¹Published artifact POM: <https://repo.maven.apache.org/maven2/net/sourceforge/owlapi/org.semanticweb.hermit/1.4.5.519/org.semanticweb.hermit-1.4.5.519.pom>.

Table 17: Within-block candidate/comparator ratios: median [minimum, maximum] over three pairs. Values below one favor the candidate. Ranges describe the measured pairs and are not confidence intervals.

N	Comparator	Task ratio [range]	Process ratio [range]
100	b1254c4	0.985908 [0.978469, 1.077540]	0.984859 [0.969318, 1.115056]
100	HermiT	0.046429 [0.044448, 0.049022]	0.379001 [0.361144, 0.415761]
1,000	b1254c4	0.965185 [0.957579, 0.990767]	0.935610 [0.919415, 1.003251]
1,000	HermiT	0.262592 [0.259343, 0.263180]	0.475576 [0.472795, 0.480711]
10,000	b1254c4	0.950003 [0.716227, 0.994829]	0.950974 [0.724435, 1.003096]
10,000	HermiT	0.378055 [0.372716, 0.400591]	0.415056 [0.410029, 0.437214]

I.3 Observed computational value and its scope

The candidate/HermiT paired median task ratios are 0.046429, 0.262592 and 0.378055 as N increases; the corresponding full-process ratios are 0.379001, 0.475576 and 0.415056. Table 17 reports the complete observed ranges. HermiT loading dominates its smallest case, while the advantage also persists in the complete process measurements. Relative to b1254c4, the candidate’s paired task ratios are 0.985908, 0.965185 and 0.950003. Most of its advantage over this HermiT artifact therefore predates the new follow-up optimizations. Three blocks describe measured variation; they do not establish a broad statistical population claim. Within-block ratios need not equal ratios of marginal medians.

This demonstrates a useful implementation/task pairing with existential axioms, independently of ontology prevalence. It does not prove a new asymptotic bound, superiority over other EL or OWL implementations, or termination/completeness on arbitrary recursive DLP L3 inputs. Raw phase timings, exact-answer hashes, protocol and semantic controls are preserved in `benchmarks/owl-bridge-results.json`, `benchmarks/owl-bridge-results-protocol.json` and `benchmarks/owl-bridge-smoke.json`. The table generator checks worker pairing, phase sums, answer hashes, protocol identity, marginal summaries and every stored paired ratio before rendering these tables.

J Comparing the original Bach examples

This comparison returns to the thesis’s worked examples without enlarging them or adding historical assertions. It distinguishes the complete ontology in Table 2.5, p. 35, from the separate family maintenance example in Figure 6.2, p. 150, Table 6.1, p. 151, and Example 6.3.2, pp. 158–159 [31]. The complete ontology uses the reconstruction’s DLP L3 mode; the family graph requires only its positive L0 rules. The examples’ small size makes them useful semantic tests, while limiting performance conclusions.

J.1 Inputs, questions, and independent expectations

The complete ontology retains T1–T17 and A1–A15. The displayed A16 is an ellipsis and contributes no assertions. The supplied Turtle and RDF/XML documents are checked for graph equivalence. The OWL comparisons use the same RDF/XML bytes and 25 questions, covering named individuals, named relationships, class descriptions, and schema properties. The current implementation and the preserved b1254c4 implementation are compared with the pinned HermiT distribution described in Appendix I. ZodiacEdge does not provide the complete ontology’s existence and constraint semantics through the positive-rule interface evaluated here, so it has no full-ontology comparison.

The query expectations preserve the distinctions developed in the thesis. Johann Ambrosius is a father because he is a man with a male child, although that child is unnamed. The child is not thereby identified with Johann Sebastian. Wilhelm Friedemann is recorded as a person; importing historical knowledge to classify him as a man would change the answers to the gender-qualified child questions. Marriage assertions alone do not imply membership in the one-way definitions of *Wife* or *Husband*. Empty named answers for birthday events do not deny the events required to exist. Negative answers consistently mean absence of entailment, not entailed negation. Generated witness identities and internal fact counts are not cross-system answer criteria.

The reconstruction retains the documented scope decisions for Table 2.5 T12 and Example 5.4.2, pp. 133–134. The latter question explicitly applies its person and gender conditions to the child. The father satisfiability question does not imply that every possible fresh class assertion terminates under the same procedure; the mutual existence requirements of *Wife* and *Husband* remain outside such a conclusion.

The family input has nine asserted ancestor links and two initial rules: ancestor transitivity and inclusion of ancestry in dynasty. Its nine questions consist of the seven existing source-based questions and retrieval of both complete named

relations. The three changes independently replace one ancestor assertion, remove the ancestor-to-dynasty rule, or add dynasty symmetry. Each begins with the original input. Each incremental sequence also reverses its change, providing a check that the original answers are restored.

Independent graph traversal establishes the expected relations. The original state has 24 ancestor and 24 dynasty pairs. Replacing the Johann Sebastian–Wilhelm Friedemann assertion by the Johann Sebastian–Johann Christian assertion gives 25 pairs in each relation. Removing the inclusion rule leaves 24 ancestor pairs and no dynasty pairs. Adding dynasty symmetry leaves ancestry unchanged and produces 48 dynasty pairs. These are expected logical counts, not internal storage counts. Every answer set is compared in full, rather than accepted from its cardinality alone.

Two source corrections matter directly. The insertion notation on p. 159 is resolved by its preceding prose and resulting deltas. Symmetry in the p. 151 example reverses ancestry pairs but does not connect the separate Johann Christoph and Christoph branches. In particular, Johannes remains an ancestor of Wilhelm Friedemann after the fact replacement because the path through Maria Barbara survives. The thesis presents this retraction as a hypothetical change to recorded knowledge, not a historical discovery about parentage.

Schema questions are not answered from accidental properties of the observed relations. In the original state, dynasty happens to contain the same pairs as ancestry, although its transitivity is not entailed. ZodiacEdge evaluates these questions through fresh canonical assertions and the active positive rules. The isolated rule evaluation tests the required implication; it does not inspect the existing relation for apparent symmetry or transitivity. Preparation and reasoning for these probes are charged to query time.

J.2 Comparison matrix and timing boundaries

The recorded design specifies twelve blocks of 28 fresh processes, giving 336 processes. Each block contains three complete-ontology runs, sixteen family reconstructions, and nine incremental sequences. The reconstructions cover four states in each of four systems. The sequences cover three changes in the current implementation, the preserved baseline, and ZodiacEdge. HermiT reconstructs the four family states; no incremental HermiT measurement is claimed. System order rotates within blocks and benchmark processes run sequentially. The protocol records the inputs, implementations, order, and hash seeds before the timed series.

These are new observations of the preserved baseline implementation. Its original saved benchmark report contained no Bach cases. The current source is archived separately, and neither new measurement replaces earlier results. Likewise, the earlier university rule-program comparison and the synthetic existential HermiT comparison retain their original inputs and outcomes.

Static task time sums input and query preparation, conversion where required, reasoner construction, consistency when supplied, and all fully consumed answers. Phase times are retained because implementations defer different amounts of work to queries. Query-specific reconstruction required by a local API is included. Incremental task time includes preparation and application of the change followed by the nine updated answers; original-state setup is separate. The reverse change is also recorded and checked. ZodiacEdge’s mixed fact replacement uses its separate deletion and insertion calls, with both included and no intervening query. This sequence is not described as an atomic mixed update.

An independent outer measurement covers the full process, including runtime startup, native bootstrap, checking, serialization, and shutdown. Source preparation and Java compilation are outside the timed series. Process restarts do not purge filesystem caches. The task intervals are reported as defined phase sums, rather than assumed to account for every part of the outer measurement. Absolute times are important on these small inputs, and the comparison makes no claim about scaling to larger genealogies.

J.3 Correctness outcomes and an external update discrepancy

Separate diagnostic execution of the pinned ZodiacEdge version exposed a loss of surviving support during fact replacement in some repeated executions. The affected updated ancestor and dynasty relations omit the Johannes–Wilhelm Friedemann pair despite the intact path through Maria Barbara. The corresponding changed state has an independent graph-traversal oracle and can also be evaluated from its assertions. This separates an incremental-state discrepancy from ambiguity in the intended logical answer. The observation is scoped to the tested version, input, and operation; it does not establish the behaviour of other versions or all incremental methods.

The retained direct-API diagnostic in `benchmarks/bach-native-deletion-diagnostic.json` reproduces the loss without using the reconstructed reasoner, its ontology compiler, or the comparison’s schema-query probes. Native iteration can depend on object identities as well as hash seeds, so selecting a seed does not guarantee the same outcome

Table 18: Dissertation Bach examples, twelve repeated blocks. Median complete-task milliseconds, including all 25 full-ontology or nine family questions. Rebuilds include input preparation; updates exclude original-state setup. A failed configuration/task receives no timing ranking.

Task	Earlier baseline	Current	HermiT	ZodiacEdge
Rebuild: Full ontology	89.52	55.56	347.37	–
Rebuild: Original family	9.41	7.49	269.67	4.43
Rebuild: Fact replacement	9.44	7.66	269.04	4.53
Rebuild: Rule removal	8.47	6.75	267.74	4.14
Rebuild: Symmetry addition	10.58	6.77	277.84	5.15
Update: Fact replacement	5.85	4.85	–	Failed (6/12)
Update: Rule removal	4.77	4.00	–	0.47
Update: Symmetry addition	6.53	3.64	–	1.27

Table 19: Fresh-process wall time for the original Bach tasks, milliseconds. These intervals additionally include runtime startup, validation, serialization, and shutdown. Filesystem caches were not cleared.

Static task	Earlier baseline	Current	HermiT	ZodiacEdge
Full ontology	220.6	184.1	503.7	–
Original family	136.6	135.6	416.3	137.9
Fact replacement	136.4	134.8	416.8	138.8
Rule removal	135.3	134.4	409.7	139.3
Symmetry addition	137.0	135.7	422.1	141.5

in another process. The diagnostic and formal-run failure counts describe the recorded executions; they are not estimates of a general failure probability.

Failed executions, their diagnostics, and any answers emitted before failure remain in the raw record. Any failed execution disqualifies its complete system–task combination from timing rankings, rather than leaving a median computed from selected successful executions. Unsupported comparisons are reported separately from wrong answers or failures to complete. The earlier successful checks on ZodiacEdge’s university rule changes remain evidence about those particular tasks, not a guarantee that every other change is handled correctly.

Raw observations and the frozen protocol are retained in `benchmarks/bach-external-results.json` and `benchmarks/bach-external-results-protocol.json`. The candidate source and actual generated inputs are preserved separately in `benchmarks/bach-external-source.tar.gz` and `benchmarks/bach-external-inputs.tar.gz`. The paper preparation step checks the answer contracts, pairing, and timing eligibility before producing the result table and records the evidence files’ digests. Rebuilding the manuscript does not rerun the experiments.

K Final default configuration and baseline validation

The production configuration now enables monotone validation/plan reuse, incremental survivor-index maintenance, and positional positive joins. It also includes the recursive-closure lifecycle correction. Adaptive unary planning, shared support certificates, and the new lookahead remain disabled. The final package is preserved in the later lookahead source archive; it differs from the primary follow-up archive, whose switches were enabled at runtime and which predates the lifecycle correction.

The complete source passes 741 tests, lint, both independent thesis counterexample scripts, the original research-artifact validator, and a source and wheel build. The final thesis suite contains 54 fresh case workers, each with five observations: all 270 observations pass the workload’s checks. All 51 historical cases match the fixed b1254c4 input hashes and correctness controls. The three Bach cases also pass but lie outside that baseline’s coverage. Recorded package hashes match the frozen later archive; source integrity is checked throughout the suite.

All 51 recorded materialization medians are lower than the saved baseline in this run, with ratios ranging from 0.00791 to 0.83264. The very large factored expression and schema-query differences include the preceding modernization; they cannot be assigned to the latest three changes. These observations also include naive and OWL-RL controls, and historical runs were neither interleaved nor isolated from host drift. The result is a longitudinal regression check, not a causal effect estimate or external-system ranking.

Table 20: Final default configuration, selected historical medians in milliseconds (materialization unless specified). Five observations per case, sharing one worker. The ratio is final/b1254c4; these longitudinal observations are not an interleaved comparison.

Workload/operation	b1254c4	5531be4	Final	Ratio
Unions, 64 pairs	10,406.030	105.388	82.267	0.008
Enumerations, 64 pairs	95.863	5.717	4.358	0.045
Taxonomy D7/I15/PF	4,860.310	4,777.663	3,336.927	0.687
Same: subsumption	7,983.876	12.843	9.338	0.001
Transitive, 100	451.250	446.153	317.792	0.704
Cardinality D5/I3	243.571	227.536	166.633	0.684
D5/10% rule insert	219.359	228.417	225.281	1.027
D5/10% rule replace	417.815	457.058	200.632	0.480
D5/10% rule delete	685.817	413.883	181.603	0.265
D5/10% atomic mixed	771.007	791.637	284.407	0.369
D4/15% fact insert	24.074	24.232	30.287	1.258

Maintenance is not uniformly faster. Of the 36 case/operation comparisons, 34 have lower update medians, while D4/15% fact insertion is 25.8% slower and D5/10% rule insertion is 2.7% slower than the saved baseline. These costs are retained in the table and raw data. The matched external experiments give more focused attribution evidence than these sequential historical comparisons. The final observations are stored separately in `benchmarks/followup-thesis-results.json`; neither the original baseline nor the preceding production results are overwritten.

L Are DLP fragments still worthwhile alongside OWL 2 RL?

Our answer separates theoretical and computational merit from practical adoption. A fragment can be worthwhile even when its prevalence in deployed ontologies is unknown. DLP remains useful as a foundation and as a decomposition of evaluator capabilities. OWL 2 RL explicitly credits DLP among its influences [18]; standardization incorporated the central idea rather than making it irrelevant.

In particular, DLP L3’s existential consequents can express useful statements beyond RL’s syntax. If specialized DLP L3 reasoning is faster than a more expressive OWL reasoner on the same DLP L3 inputs and reasoning tasks, while preserving the same correctness and completeness obligations, that is evidence for a useful computational fragment. No estimate of how often users need only DLP L3 is required for this conclusion. The finite existential-bridge study in Appendix I now supplies such scoped computational evidence: the preserved baseline and candidate both answer the same consistency and named-instance task faster than the pinned broader-language HermiT distribution [6]. All 27 measured workers match independently justified answers. The candidate/HermiT paired median task ratios are 0.046429, 0.262592, and 0.378055 for 100, 1,000, and 10,000 named A instances; the corresponding full-process ratios are 0.379001, 0.475576, and 0.415056. These are results for one controlled family and three process blocks, not a new complexity bound or a comparison with every broader-language OWL implementation. Most of this advantage is already present in the fixed baseline.

Measured advantages and mathematical guarantees provide different evidence. One can prove an expressiveness–complexity tradeoff without an adoption survey; one can also demonstrate an implementation advantage without proving a new complexity separation. Hustadt, Motik, and Sattler [14] establish P-complete data complexity for basic Horn-SHIQ reasoning, while general SHIQ instance checking is coNP-complete under their assumptions. This is an example of prior theoretical justification for expressive Horn fragments. It does not identify our DLP L3 with Horn-SHIQ or give our unrestricted witness chase that bound. Termination, combined complexity, data complexity, and query complexity require their own precise language and task definitions. In particular, nontermination of this Skolem chase is a limitation of the chosen procedure, not by itself a proof that reasoning in the original description-logic fragment is undecidable.

Three classifications must remain distinct: the thesis’s constructor-position fragments, the actual RDF encodings accepted by this compiler, and the conditions under which an optimization is valid. An L0 ontology can contain expensive transitive binary joins. An ontology accepted with the L2 ceiling may contain only unary implications. Choosing a higher ceiling does not add absent equality or constraints, and a label alone cannot predict cost.

L.1 Overlap rather than automatic profile inclusion

Concrete syntax witnesses rule out identifying these labels with RL subsets. The DLP L3 inclusion $A \sqsubseteq \exists R.B$ has a general existential consequent absent from RL’s superclass grammar. Even L1’s $\top \sqsubseteq A$ is outside its subclass grammar. Conversely, RL supports restricted property chains and keys, which this compiler rejects [18]. These are statements about syntactic membership, not impossibility of equivalent rewriting or a claim that DLP L3 contains RL.

Global restrictions matter independently. The compiler accepts simultaneous transitivity and functionality of the same property. That rule set has a first-order interpretation, but the property fails OWL 2 DL’s simplicity condition for functionality [20]. Parser acceptance therefore does not certify a standard OWL 2 RL ontology. Datatype completeness is another separate gap in this implementation.

OWL 2 RL/RDF rules can also be applied beyond syntactically conforming RL. Their Direct-Semantics correspondence has explicit conditions, including restrictions on entity roles and on the conclusions covered by the theorem [18, 29]. A comparison must choose its semantic contract. Our equality acts on individual arguments, not predicate identifiers. Requiring identity with every triple produced by an RDF-Based reasoner can therefore demand consequences outside the intended contract.

L.2 What restrictions can still buy

For this implementation, function freedom gives finite active-domain materialization. Eligible states without explicit or datatype-induced equality avoid representative splitting during deletion. Same-variable unary bodies permit set-intersection evaluation. These properties are useful internal dispatch conditions, including within a broader supported ontology. They do not require presenting another public ontology language as a replacement for RL.

A small compiler can also be easier to audit and reproduce. Its value depends on real schemas fitting the contract and on explicit rejection of unsupported axioms. Conversion costs, excluded standard features, and additional conformance work are part of the cost of the restriction. The earlier certificate and unary experiments demonstrate workload-specific benefits and failures, not an advantage of requiring a DLP frontend over accepting the same axioms through a conforming RL frontend.

LUBM illustrates the distinction between ontology coverage and query sufficiency. Its full ontology needs this implementation’s DLP L3 mode, and all fourteen named answer sets match. That does not prove those questions require generated witnesses. Nor would matching them after a projection prove ontology equivalence. Existential consequences can affect named answers: from $A(a)$, $A \sqsubseteq \exists R.B$, and $\exists R.B \sqsubseteq C$, we obtain $C(a)$ [19]. Dropping the witness-producing inclusion without a justified rewriting loses this consequence. The HermiT study evaluates precisely this bridge together with a B-only negative individual and an unrelated empty-answer class. Both GCIs are essential to the answer, but explicit witness construction is not: the TBox entails $A \sqsubseteq C$, and HermiT retains its normal optimizations. The family also belongs to OWL 2 EL, so the result does not establish superiority over specialized EL reasoners. It compares the specified entailment task, not entire materializations or generated witness identities.

L.3 Practical adoption is a separate question

OWL 2 EL and QL supply other established tradeoffs, respectively oriented toward large terminologies with existential descriptions and query rewriting over large instance data [10]. They are alternatives to evaluate, not automatic solutions for every DLP L3 query. In particular, avoiding explicit infinite witness expansion for classification does not establish completeness for arbitrary conjunctive queries or cardinality constraints.

The frequency with which real applications need only DLP L3 is a difficult empirical question. Published ontology collections can reveal supported constructs in those collections, but may be unrepresentative; they also do not fully reveal application queries, update needs, or equivalent rewritings. We do not require an answer to that question before recognizing a fragment’s theoretical value.

For practical interoperability claims, checked standard profiles and explicit extensions would complement the historical DLP L0–L3 interface. A deployment study could measure coverage and rejection reasons, full input-to-answer and update costs, conversion and memory, and ablations with compiled rules held fixed. Its coverage conclusions would remain specific to the sampled population. Current evidence supports the fragments as foundations and research targets and demonstrates a computational advantage on the stated existential task. That scoped result stands independently of prevalence; it does not establish the historical interface as a better interoperability standard or settle performance on other ontology and query families.

M Reproducibility and provenance

The original thesis file has SHA-256

```
80146ec324e0a7c342508e38cbe7712f207db8917d6655cd70b9acbaee2f570
```

and the pinned b1254c4 result file has SHA-256

```
6f0bc493cae0fb96b474405eb9496dc9ce2c3155b313f2bcd3de49b3dc8b51e2.
```

The source archive, protocols, raw primary observations, post-hoc replication, and final production observations are distinct files. Their measurement boundaries must remain distinct when recomputing summaries. Tables of our new benchmark measurements are generated from these JSON artifacts. Published reference values are separately attributed to their sources. The preparation script records input digests.

The principal existing checks and reproduction commands are:

```
uv sync --extra dev --locked
uv run pytest -q
uv run ruff check src tests benchmarks scripts
uv run python scripts/check_thesis_logic.py
uv run python scripts/check_thesis_errata.py
uv run python scripts/validate_research_artifacts.py
uv run python scripts/replay_support_research.py \
  --output /tmp/dlp-support-reproduction.json
uv run python -m benchmarks.run --suite thesis --repeats 5 \
  --output /tmp/dlp-thesis-reproduction.json
```

Heavy timed reproductions should run sequentially. They record their own source and runtime state and are not replacements for the archived observations. The raw primary worker assertion/rule digests describe the expected candidate; the integration tests independently check the actual internal candidate state. The post-hoc replication checks original-report, driver, protocol, and source hashes and uses seeds 3000–3008.

The manuscript uses George Kour’s MIT-licensed arxiv-style template, pinned by commit and file digest in `template-provenance.json`. This is a community preprint style, not an official arXiv endorsement. The source package includes the local style and compiled bibliography, uses standard pdfLaTeX packages, requires no shell escape, and can be rebuilt without benchmark execution or network access. The repository’s paper README documents the build, external dataset preparation, and packaging commands.

M.1 Data and research transparency

The accompanying research artifact contains the code and saved experimental observations; the [project repository](#) provides the development history. The fixed reference results and earlier measurements are preserved. The Bach comparisons add new observations of the thesis’s original examples, including fresh measurements of the preserved reference implementation. AI assistance was used in implementation, literature retrieval, analysis, and manuscript preparation.

References

- [1] Güneş Aluç, Olaf Hartig, M. Tamer Özsu, and Khuzaima Daudjee. Diversified stress testing of RDF data management systems. In *Proceedings of ISWC*, pages 197–212, 2014. doi: 10.1007/978-3-319-11964-9_13. URL <https://cs.uwaterloo.ca/~tozsu/publications/rdf/ISWC14-final.pdf>.
- [2] Labinot Bajraktari, Magdalena Ortiz, and Mantas Šimkus. Goal-oriented type-based reasoning for expressive DLs. In *Description Logics*, volume 1879 of *CEUR Workshop Proceedings*, 2017. URL <https://ceur-ws.org/Vol-1879/paper63.pdf>. Paper 63, extended abstract.
- [3] Liese Bekkers, Frank Neven, Stijn Vansummeren, and Yisu Remy Wang. Instance-optimal acyclic join processing without regret: Engineering the yannakakis algorithm in column stores. *Proceedings of the VLDB Endowment*, 18(8):2413–2426, 2025. doi: 10.14778/3742728.3742737. URL <https://www.vldb.org/pvldb/vol18/p2413-vansummeren.pdf>.
- [4] Christian Bizer and Andreas Schultz. The Berlin SPARQL benchmark. *International Journal on Semantic Web and Information Systems*, 5(2):1–24, 2009. doi: 10.4018/jswis.2009040101.
- [5] David Carral, Irina Dragoste, Larry González, Cerial Jacobs, Markus Krötzsch, and Jacopo Urbani. VLog: A rule engine for knowledge graphs. In *Proceedings of ISWC*, volume 11779 of *LNCS*, pages 19–35, 2019. doi: 10.1007/978-3-030-30796-7_2. URL <https://iccl.inf.tu-dresden.de/web/Inproceedings3218/en>.
- [6] Birte Glimm, Ian Horrocks, Boris Motik, Giorgos Stoilos, and Zhe Wang. HermiT: An OWL 2 reasoner. *Journal of Automated Reasoning*, 53(3):245–269, 2014. doi: 10.1007/s10817-014-9305-1. URL <https://www.cs.ox.ac.uk/people/boris.motik/pubs/ghmsw14HermiT.pdf>.
- [7] Benjamin N. Groszof, Ian Horrocks, Raphael Volz, and Stefan Decker. Description logic programs: Combining logic programs with description logic. In *Proceedings of WWW*, pages 48–57, 2003. doi: 10.1145/775152.775160. URL <https://www.cs.ox.ac.uk/people/ian.horrocks/Publications/download/2003/p117-groszof.pdf>.
- [8] Yuanbo Guo, Zhengxiang Pan, and Jeff Heflin. LUBM: A benchmark for OWL knowledge base systems. *Journal of Web Semantics*, 3(2–3):158–182, 2005. doi: 10.1016/j.websem.2005.06.005. URL <https://swat.cse.lehigh.edu/pubs/guo05a.pdf>.
- [9] Ashish Gupta, Inderpal Singh Mumick, and V. S. Subrahmanian. Maintaining views incrementally. In *Proceedings of SIGMOD*, pages 157–166, 1993. doi: 10.1145/170035.170066.
- [10] Pascal Hitzler, Markus Krötzsch, Bijan Parsia, Peter F. Patel-Schneider, and Sebastian Rudolph. OWL 2 web ontology language primer (second edition), 2012. URL <https://www.w3.org/TR/owl2-primer/>. W3C Recommendation, 11 December 2012; Section 10.
- [11] Pan Hu, Boris Motik, and Ian Horrocks. Optimised maintenance of Datalog materialisations. In *Proceedings of AAAI*, pages 1871–1879, 2018. doi: 10.1609/aaai.v32i1.11554. URL <https://www.cs.ox.ac.uk/people/ian.horrocks/Publications/download/2018/HuMH18a.pdf>.
- [12] Pan Hu, Jacopo Urbani, Boris Motik, and Ian Horrocks. Datalog reasoning over compressed RDF knowledge bases. In *Proceedings of CIKM*, pages 2065–2068, 2019. doi: 10.1145/3357384.3358147. URL <https://arxiv.org/abs/1908.10177>.
- [13] Pan Hu, Boris Motik, and Ian Horrocks. Modular materialisation of Datalog programs. *Artificial Intelligence*, 308:103726, 2022. doi: 10.1016/j.artint.2022.103726. URL <https://www.cs.ox.ac.uk/people/boris.motik/pubs/hmh22modular-materialisation.pdf>.
- [14] Ullrich Hustadt, Boris Motik, and Ulrike Sattler. Data complexity of reasoning in very expressive description logics. In *Proceedings of the Nineteenth International Joint Conference on Artificial Intelligence*, pages 466–471, 2005. URL <https://www.cs.ox.ac.uk/people/boris.motik/pubs/hms05data.pdf>.
- [15] Jakub Kotowski, François Bry, and Simon Brodt. Reasoning as axioms change: Incremental view maintenance reconsidered. In *Web Reasoning and Rule Systems*, volume 6902 of *LNCS*, pages 139–154, 2011. doi: 10.1007/978-3-642-23580-1_11. URL <https://www.en.pms.ifi.lmu.de/publications/PMS-FB/PMS-FB-2011-5/PMS-FB-2011-5-paper.pdf>.
- [16] Lehigh University SWAT Project. Lehigh university benchmark: UBA 1.7, ontology, queries, and answers, 2026. URL <https://swat.cse.lehigh.edu/projects/lubm/>. Artifact page accessed 9 September 2026.
- [17] Li Ma, Yang Yang, Zhaoming Qiu, Guotong Xie, Yue Pan, and Shengping Liu. Towards a complete OWL ontology benchmark. In *Proceedings of ESWC*, volume 4011 of *LNCS*, pages 125–139, 2006. doi: 10.1007/11762256_12.
- [18] Boris Motik, Bernardo Cuenca Grau, Ian Horrocks, Zhe Wu, Achille Fokoue, and Carsten Lutz. OWL 2 web ontology language profiles (second edition), 2012. URL <https://www.w3.org/TR/2012/REC-owl2-profiles-20121211/>. W3C Recommendation, 11 December 2012; Sections 4–5.
- [19] Boris Motik, Peter F. Patel-Schneider, and Bernardo Cuenca Grau. OWL 2 web ontology language direct semantics (second edition), 2012. URL <https://www.w3.org/TR/owl2-direct-semantics/>. W3C Recommendation, 11 December 2012; Tables 4–5.
- [20] Boris Motik, Peter F. Patel-Schneider, and Bijan Parsia. OWL 2 web ontology language structural specification and functional-style syntax (second edition), 2012. URL <https://www.w3.org/TR/owl2-syntax/>. W3C Recommendation, 11 December 2012; Section 11.
- [21] Boris Motik, Yavor Nenov, Robert Piro, Ian Horrocks, and Dan Olteanu. Parallel materialisation of Datalog programs in centralised, main-memory RDF systems. In *Proceedings of AAAI*, pages 129–137, 2014. doi: 10.1609/aaai.v28i1.8730. URL <https://ojs.aaai.org/index.php/AAAI/article/view/8730>.
- [22] Boris Motik, Yavor Nenov, Robert Piro, and Ian Horrocks. Incremental update of Datalog materialisation: The backward/forward algorithm. In *Proceedings of AAAI*, pages 1560–1568, 2015. doi: 10.1609/aaai.v29i1.9409. URL

- <https://ojs.aaai.org/index.php/AAAI/article/view/9409>.
- [23] Boris Motik, Yavor Nenov, Robert Piro, and Ian Horrocks. Maintenance of Datalog materialisations revisited. *Artificial Intelligence*, 269:76–136, 2019. doi: 10.1016/j.artint.2018.12.004. URL <https://www.cs.ox.ac.uk/people/ian.horrocks/Publications/download/2019/MNPH19a.pdf>.
 - [24] Hung Q. Ngo, Ely Porat, Christopher Ré, and Atri Rudra. Worst-case optimal join algorithms, 2012. URL <https://arxiv.org/abs/1203.1952>. arXiv:1203.1952v1.
 - [25] Bijan Parsia, Nicolas Matentzoglou, Rafael S. Gonçalves, Birte Glimm, and Andreas Steigmiller. The OWL reasoner evaluation (ORE) 2015 competition report. *Journal of Automated Reasoning*, 59:455–482, 2017. doi: 10.1007/s10817-017-9406-8.
 - [26] Peter F. Patel-Schneider, Patrick Hayes, and Ian Horrocks. OWL web ontology language semantics and abstract syntax, 2004. URL <https://www.w3.org/TR/2004/REC-owl-semantic-20040210/direct.html>. W3C Recommendation, 10 February 2004; Section 3.
 - [27] Yiming Qiao, Peter Boncz, and Huanchen Zhang. Robust predicate transfer with dynamic execution. *Proceedings of the VLDB Endowment*, 19(6):1278–1290, 2026. doi: 10.14778/3797919.3797934. URL <https://www.vldb.org/pvldb/vol19/p1278-qiao.pdf>.
 - [28] Gunjan Singh, Sumit Bhatia, and Raghava Mutharaju. OWL2Bench: A benchmark for OWL 2 reasoners. In *Proceedings of ISWC*, volume 12507 of *LNCS*, pages 81–96, 2020. doi: 10.1007/978-3-030-62466-8_6. URL <https://github.com/kracr/owl2bench>.
 - [29] Michael Smith, Ian Horrocks, and Markus Krötzsch. OWL 2 web ontology language conformance (second edition), 2012. URL <https://www.w3.org/TR/owl2-conformance/>. W3C Recommendation, 11 December 2012; Section 2.2.
 - [30] Transaction Processing Performance Council. TPC-H: A decision support benchmark, 2026. URL <https://www.tpc.org/tpch/>. Official benchmark description; accessed 9 September 2026.
 - [31] Raphael Volz. *Web Ontology Reasoning with Logic Databases*. PhD thesis, Universität Karlsruhe (TH), 2004. URL <https://publikationen.bibliothek.kit.edu/162004>.
 - [32] Raphael Volz, Steffen Staab, and Boris Motik. Incremental maintenance of materialized ontologies. In *On the Move to Meaningful Internet Systems*, volume 2888 of *LNCS*, pages 707–724, 2003. doi: 10.1007/978-3-540-39964-3_45. URL <https://www.cs.ox.ac.uk/boris.motik/pubs/vsm03incremental.pdf>.
 - [33] Weiqin Xu and Olivier Curé. ZodiacEdge: A Datalog engine with incremental rule set maintenance, 2023. URL <https://arxiv.org/abs/2312.14530>. arXiv:2312.14530v1, 22 December 2023.
 - [34] Weiqin Xu and Olivier Curé. ZodiacEdge repository: Additional LUBM rule-maintenance evaluation, 2026. URL https://github.com/xwq610728213/zodiac_edge/blob/e15721161d1055f019c4b8f9ac1bd8648417f154/README.md. Undated repository results; inspected 9 September 2026. Commit e15721161d1055f019c4b8f9ac1bd8648417f154.
 - [35] Xinyue Zhang, Pan Hu, Yavor Nenov, and Ian Horrocks. Optimised storage for Datalog reasoning. In *Proceedings of AAAI*, volume 38, pages 10748–10755, 2024. doi: 10.1609/aaai.v38i9.28947. URL <https://arxiv.org/abs/2312.11297>.